

Cloud-Native Healthcare Data Management System

Phase 2 — Complete Source Code

BITS Pilani · Cloud Computing (CSIZG527) · Group 23
Karan Rawat · Vikas Kumar · Kriti Tripathi · Avanish Pratap Singh

app/___init___.py

app/auth.py

```
from datetime import datetime, timedelta, timezone
from typing import Optional

from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from sqlalchemy.orm import Session

from app.config import settings
from app.database import get_db
from app.models.user import User

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
security = HTTPBearer()

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (expires_delta or timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)

def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security),
    db: Session = Depends(get_db),
) -> User:
    """Decode JWT and return the current User. Raises 401 on failure."""
    token = credentials.credentials
    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
        username: str = payload.get("sub")
        if username is None:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid token")
    except JWTError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid token")

    user = db.query(User).filter(User.username == username).first()
    if user is None:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="User not found")
    return user

def require_roles(*roles: str):
    """Dependency factory: ensures current user has one of the allowed roles."""
    def role_checker(current_user: User = Depends(get_current_user)):
        if current_user.role not in roles:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Insufficient permissions")
        return current_user
    return role_checker
```

app/config.py

```
from pydantic_settings import BaseSettings
from pathlib import Path

class Settings(BaseSettings):
    # App
    APP_NAME: str = "Healthcare DMS"
    DEBUG: bool = True

    # Database - SQLite for Phase 1, swap to PostgreSQL URL for Phase 2
    DATABASE_URL: str = "sqlite:///./health_dms.db"

    # JWT Auth
    SECRET_KEY: str = "dev-secret-key-change-in-production"
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60

    # File uploads
    UPLOAD_DIR: str = "./uploads"
    MAX_UPLOAD_SIZE_MB: int = 10
    ALLOWED_EXTENSIONS: set = {".pdf", ".jpg", ".jpeg", ".png"}

    # Phase 2 - AWS storage backend
    # STORAGE_BACKEND: "local" (Phase 1) or "s3" (Phase 2)
    STORAGE_BACKEND: str = "local"
    S3_BUCKET: str = ""
    S3_PREFIX: str = "medical-records/"
    S3_KMS_KEY_ID: str = "" # optional KMS CMK ARN for SSE-KMS
    AWS_REGION: str = "ap-south-1"

    # Phase 2 - SNS topic for appointment notifications (optional)
    SNS_TOPIC_ARN: str = ""

    # Phase 2 - structured (CloudWatch-friendly) JSON logging
    JSON_LOGS: bool = False
    LOG_LEVEL: str = "INFO"

    model_config = {"env_file": ".env", "extra": "ignore"}

settings = Settings()

# Ensure upload directory exists when running with the local backend
if settings.STORAGE_BACKEND.lower() == "local":
    Path(settings.UPLOAD_DIR).mkdir(parents=True, exist_ok=True)
```

app/database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase

from app.config import settings

# SQLite needs check_same_thread=False; PostgreSQL doesn't use this
connect_args = {}
if settings.DATABASE_URL.startswith("sqlite"):
    connect_args = {"check_same_thread": False}

engine = create_engine(settings.DATABASE_URL, connect_args=connect_args)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass

def get_db():
    """Dependency that provides a DB session per request."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

app/lambda_handlers/__init__.py

```
"""AWS Lambda handlers (Phase 2 serverless compute layer)."""
```

app/lambda_handlers/appointment_reminder.py

```
"""AWS Lambda handler – appointment reminders.
```

```
Architecture (Phase 2):
```

```
CloudWatch Events (cron, every 15 min)
```



```
AWS Lambda (this module)
```

```
■■■■ reads appointments due in the next 24h from RDS PostgreSQL
```

```
■■■■ publishes one message per appointment to SNS topic
```



```
SNS Topic ■■■■ Email / SMS subscribers (patient contacts, hospital staff)
```

```
Why Lambda?
```

- * Serverless – no idle cost, scales to N concurrent invocations.
- * Event-driven – fires only when CloudWatch Events triggers it.
- * Aligns with Project Objective 01 (cloud-native serverless compute).

```
Local execution:
```

```
python -m app.lambda_handlers.appointment_reminder  
(uses settings.DATABASE_URL, defaults to local SQLite)
```

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
import logging
```

```
import os
```

```
from datetime import datetime, timedelta, timezone
```

```
from typing import Any
```

```
logger = logging.getLogger("lambda.appointment_reminder")  
logger.setLevel(logging.INFO)
```

```
def _build_message(appointment: Any) -> dict:
```

```
    """Build the SNS message payload for one appointment."""
```

```
    return {  
        "appointment_id": appointment.id,  
        "patient_id": appointment.patient_id,  
        "doctor_id": appointment.doctor_id,  
        "scheduled_for": appointment.date_time.isoformat(),  
        "reason": appointment.reason,  
        "channel": "reminder",  
    }
```

```
def _publish_to_sns(sns_client, topic_arn: str, message: dict) -> str | None:
```

```
    """Publish to SNS; returns MessageId or None if no topic configured."""
```

```
    if not topic_arn:  
        logger.info("SNS topic not configured – skipping publish (dry run)")  
        return None  
    resp = sns_client.publish(  
        TopicArn=topic_arn,  
        Subject="Healthcare DMS – Appointment Reminder",  
        Message=json.dumps(message),  
    )  
    return resp.get("MessageId")
```

```
def find_due_appointments(db, hours_ahead: int = 24) -> list:
```

```
    """Query RDS for scheduled appointments inside the reminder window."""
```

```
    # Imported here so unit tests can stub the DB without importing the whole app.  
    from app.models.appointment import Appointment
```

```
    now = datetime.now(timezone.utc).replace(tzinfo=None)  
    window_end = now + timedelta(hours=hours_ahead)  
    return (  
        db.query(Appointment)  
        .filter(  
            Appointment.status == "scheduled",  
            Appointment.date_time >= now,  
            Appointment.date_time <= window_end,  
        )  
        .all()  
    )
```

```
def lambda_handler(event: dict, context: Any) -> dict:
```

```
    """AWS Lambda entry point.
```

```
    Returns:
```

```
    {"published": int, "skipped": int, "ids": [...]}.  
    """
```

```
    # Lazy imports keep cold-start small if some deps are missing in tests.  
    from app.database import SessionLocal
```

```
    topic_arn = os.environ.get("SNS_TOPIC_ARN", "")  
    hours_ahead = int(os.environ.get("REMINDER_WINDOW_HOURS", "24"))
```

```
    sns = None
```

```
    if topic_arn:  
        import boto3 # type: ignore  
  
        sns = boto3.client("sns")
```

```
    db = SessionLocal()
```

```
    published, skipped, ids = 0, 0, []
```

```
    try:
```

```
        appointments = find_due_appointments(db, hours_ahead=hours_ahead)  
        logger.info("found %d appointments in next %dh", len(appointments), hours_ahead)  
        for appt in appointments:
```

```
        msg = _build_message(appt)
        mid = _publish_to_sns(sns, topic_arn, msg)
        if mid is None:
            skipped += 1
        else:
            published += 1
            ids.append(mid)
    finally:
        db.close()

    return {"published": published, "skipped": skipped, "ids": ids}

if __name__ == "__main__": # pragma: no cover - manual execution
    print(json.dumps(lambda_handler({}, None), indent=2))
```


app/logging_middlewares.py

```
"""CloudWatch-friendly structured JSON logging.

When `settings.JSON_LOGS=True` (set on EC2 via environment), every log line
is emitted as a single JSON object that the CloudWatch agent ingests
directly into CloudWatch Logs. The middleware also stamps each request
with a UUID and records its latency, enabling per-request tracing in
CloudWatch Logs Insights.
"""
from __future__ import annotations

import json
import logging
import sys
import time
import uuid
from datetime import datetime, timezone

from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import Response

from app.config import settings

class JsonFormatter(logging.Formatter):
    """Renders log records as one-line JSON suitable for CloudWatch Logs."""
    def format(self, record: logging.LogRecord) -> str: # pragma: no cover - thin wrapper
        payload = {
            "ts": datetime.now(timezone.utc).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "msg": record.getMessage(),
        }
        # Merge any extra fields attached via logger.info("msg", extra={...})
        for key, value in record.__dict__.items():
            if key in ("args", "msg", "levelname", "name", "msecs", "created",
                      "relativeCreated", "exc_info", "exc_text", "stack_info",
                      "lineno", "funcName", "module", "pathname", "filename",
                      "thread", "threadName", "processName", "process",
                      "levelno"):
                continue
            payload[key] = value
        if record.exc_info:
            payload["exc"] = self.formatException(record.exc_info)
        return json.dumps(payload, default=str)

def configure_logging() -> None:
    """Configure root logger once at app startup."""
    handler = logging.StreamHandler(sys.stdout)
    if settings.JSON_LOGS:
        handler.setFormatter(JsonFormatter())
    else:
        handler.setFormatter(logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s"))
    root = logging.getLogger()
    root.handlers = [handler]
    root.setLevel(getattr(logging, settings.LOG_LEVEL.upper(), logging.INFO))

logger = logging.getLogger("app.request")

class RequestLoggingMiddleware(BaseHTTPMiddleware):
    """Adds an X-Request-ID header and emits one structured log per request."""
    async def dispatch(self, request: Request, call_next) -> Response:
        request_id = request.headers.get("x-request-id") or uuid.uuid4().hex
        start = time.perf_counter()
        status_code = 500
        try:
            response = await call_next(request)
            status_code = response.status_code
            return response
        finally:
            duration_ms = round((time.perf_counter() - start) * 1000, 2)
            logger.info(
                "request",
                extra={
                    "request_id": request_id,
                    "method": request.method,
                    "path": request.url.path,
                    "status": status_code,
                    "duration_ms": duration_ms,
                    "client": request.client.host if request.client else None,
                },
            )
```

app/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.config import settings
from app.database import engine, Base
from app.logging_middleware import RequestLoggingMiddleware, configure_logging
from app.routers import auth, patients, doctors, appointments, records, analytics, dashboard

# Configure JSON / plain logging based on env (CloudWatch ingests JSON in Phase 2)
configure_logging()

# Create all tables on startup
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title=settings.APP_NAME,
    description="Cloud-Native Healthcare Data Management System",
    version="2.0.0",
)

# Per-request structured logging + X-Request-ID propagation
app.add_middleware(RequestLoggingMiddleware)

# CORS – allow all in dev, restrict in production
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"] if settings.DEBUG else [],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Register routers
app.include_router(auth.router)
app.include_router(patients.router)
app.include_router(doctors.router)
app.include_router(appointments.router)
app.include_router(records.router)
app.include_router(analytics.router)
app.include_router(dashboard.router)

@app.get("/", tags=["Health"])
def health_check():
    return {"status": "healthy", "app": settings.APP_NAME}
```

app/models/__init__.py

app/models/appointment.py

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey, func
from app.database import Base

class Appointment(Base):
    __tablename__ = "appointments"

    id = Column(Integer, primary_key=True, index=True)
    patient_id = Column(Integer, ForeignKey("patients.id"), nullable=False)
    doctor_id = Column(Integer, ForeignKey("doctors.id"), nullable=False)
    date_time = Column(DateTime, nullable=False)
    reason = Column(String(200), default="")
    status = Column(String(20), default="scheduled") # scheduled, completed, cancelled
    created_at = Column(DateTime, server_default=func.now())
```

app/models/doctor.py

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey, func
from app.database import Base

class Doctor(Base):
    __tablename__ = "doctors"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=True)
    name = Column(String(100), nullable=False)
    specialization = Column(String(100), nullable=False)
    department = Column(String(100), nullable=False)
    contact = Column(String(15), nullable=False)
    created_at = Column(DateTime, server_default=func.now())
```

app/models/medical_record.py

```
from sqlalchemy import Column, Integer, String, Text, DateTime, ForeignKey, func
from app.database import Base

class MedicalRecord(Base):
    __tablename__ = "medical_records"

    id = Column(Integer, primary_key=True, index=True)
    patient_id = Column(Integer, ForeignKey("patients.id"), nullable=False)
    doctor_id = Column(Integer, ForeignKey("doctors.id"), nullable=False)
    diagnosis = Column(String(200), nullable=False)
    prescription = Column(Text, default="")
    notes = Column(Text, default="")
    file_path = Column(String(300), default="") # local path (Phase 1) or S3 key (Phase 2)
    created_at = Column(DateTime, server_default=func.now())
```

app/models/patient.py

```
from sqlalchemy import Column, Integer, String, Boolean, DateTime, func
from app.database import Base

class Patient(Base):
    __tablename__ = "patients"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    age = Column(Integer, nullable=False)
    gender = Column(String(10), nullable=False)
    contact = Column(String(15), nullable=False)
    address = Column(String(200), default="")
    blood_group = Column(String(5), default="")
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, server_default=func.now())
```

app/models/user.py

```
from sqlalchemy import Column, Integer, String, DateTime, func
from app.database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, nullable=False, index=True)
    password_hash = Column(String(128), nullable=False)
    full_name = Column(String(100), nullable=False)
    role = Column(String(20), nullable=False, default="receptionist") # admin, doctor, receptionist
    created_at = Column(DateTime, server_default=func.now())
```


app/routers/__init__.py

app/routers/analytics.py

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from sqlalchemy import func

from app.database import get_db
from app.models.patient import Patient
from app.models.doctor import Doctor
from app.models.appointment import Appointment
from app.models.user import User
from app.auth import require_roles

router = APIRouter(prefix="/analytics", tags=["Analytics"])

@router.get("/summary")
def get_summary(
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin")),
):
    total_patients = db.query(func.count(Patient.id)).filter(Patient.is_active == True).scalar()
    total_doctors = db.query(func.count(Doctor.id)).scalar()
    total_appointments = db.query(func.count(Appointment.id)).scalar()
    scheduled = db.query(func.count(Appointment.id)).filter(Appointment.status == "scheduled").scalar()
    completed = db.query(func.count(Appointment.id)).filter(Appointment.status == "completed").scalar()
    cancelled = db.query(func.count(Appointment.id)).filter(Appointment.status == "cancelled").scalar()

    # Department-wise patient count (via doctors → appointments)
    dept_stats = (
        db.query(Doctor.department, func.count(func.distinct(Appointment.patient_id)))
        .join(Appointment, Appointment.doctor_id == Doctor.id)
        .group_by(Doctor.department)
        .all()
    )

    return {
        "total_patients": total_patients,
        "total_doctors": total_doctors,
        "total_appointments": total_appointments,
        "appointments_by_status": {
            "scheduled": scheduled,
            "completed": completed,
            "cancelled": cancelled,
        },
        "patients_by_department": {dept: count for dept, count in dept_stats},
    }
```

app/routers/appointments.py

```
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from typing import Optional

from app.database import get_db
from app.models.appointment import Appointment
from app.models.patient import Patient
from app.models.doctor import Doctor
from app.models.user import User
from app.schemas.appointment import AppointmentCreate, AppointmentUpdate, AppointmentOut
from app.auth import require_roles

router = APIRouter(prefix="/appointments", tags=["Appointments"])

@router.get("/", response_model=list[AppointmentOut])
def list_appointments(
    patient_id: Optional[int] = Query(None),
    doctor_id: Optional[int] = Query(None),
    status: Optional[str] = Query(None),
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin", "doctor", "receptionist")),
):
    query = db.query(Appointment)
    if patient_id:
        query = query.filter(Appointment.patient_id == patient_id)
    if doctor_id:
        query = query.filter(Appointment.doctor_id == doctor_id)
    if status:
        query = query.filter(Appointment.status == status)
    return query.order_by(Appointment.date_time.desc()).all()

@router.post("/", response_model=AppointmentOut, status_code=201)
def create_appointment(
    body: AppointmentCreate,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin", "receptionist")),
):
    if not db.query(Patient).filter(Patient.id == body.patient_id, Patient.is_active == True).first():
        raise HTTPException(status_code=404, detail="Patient not found")
    if not db.query(Doctor).filter(Doctor.id == body.doctor_id).first():
        raise HTTPException(status_code=404, detail="Doctor not found")
    appt = Appointment(**body.model_dump())
    db.add(appt)
    db.commit()
    db.refresh(appt)
    return appt

@router.patch("/{appointment_id}", response_model=AppointmentOut)
def update_appointment_status(
    appointment_id: int,
    body: AppointmentUpdate,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin", "doctor")),
):
    if body.status not in ("scheduled", "completed", "cancelled"):
        raise HTTPException(status_code=400, detail="Invalid status")
    appt = db.query(Appointment).filter(Appointment.id == appointment_id).first()
    if not appt:
        raise HTTPException(status_code=404, detail="Appointment not found")
    appt.status = body.status
    db.commit()
    db.refresh(appt)
    return appt

@router.delete("/{appointment_id}", status_code=204)
def delete_appointment(
    appointment_id: int,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin")),
):
    appt = db.query(Appointment).filter(Appointment.id == appointment_id).first()
    if not appt:
        raise HTTPException(status_code=404, detail="Appointment not found")
    db.delete(appt)
    db.commit()
```

app/routers/auth.py

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from app.database import get_db
from app.models.user import User
from app.schemas.user import UserCreate, UserOut, Token, LoginRequest
from app.auth import hash_password, verify_password, create_access_token, require_roles

router = APIRouter(prefix="/auth", tags=["Authentication"])

@router.post("/login", response_model=Token)
def login(body: LoginRequest, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.username == body.username).first()
    if not user or not verify_password(body.password, user.password_hash):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid credentials")
    token = create_access_token(data={"sub": user.username, "role": user.role})
    return {"access_token": token}

@router.post("/register", response_model=UserOut, status_code=status.HTTP_201_CREATED)
def register(
    body: UserCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles("admin")),
):
    if body.role not in ("admin", "doctor", "receptionist"):
        raise HTTPException(status_code=400, detail="Invalid role")
    if db.query(User).filter(User.username == body.username).first():
        raise HTTPException(status_code=400, detail="Username already exists")
    user = User(
        username=body.username,
        password_hash=hash_password(body.password),
        full_name=body.full_name,
        role=body.role,
    )
    db.add(user)
    db.commit()
    db.refresh(user)
    return user

@router.get("/users", response_model=list[UserOut])
def list_users(
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin")),
):
    return db.query(User).order_by(User.id.desc()).all()

@router.get("/me", response_model=UserOut)
def me(current_user: User = Depends(require_roles("admin", "doctor", "receptionist"))):
    return current_user
```

app/routers/dashboard.py

```
from fastapi import APIRouter
from fastapi.responses import HTMLResponse

router = APIRouter(tags=["Dashboard"])

@router.get("/dashboard", response_class=HTMLResponse)
def dashboard():
    return DASHBOARD_HTML

DASHBOARD_HTML = r"""
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Healthcare DMS – Dashboard</title>
<style>
:root {
  --primary: #0f3460; --primary-light: #1a4a8a; --primary-bg: #e8eef6;
  --green: #059669; --green-bg: #d1fae5; --amber: #d97706; --amber-bg: #fef3c7;
  --red: #dc2626; --red-bg: #fee2e2; --blue: #2563eb; --blue-bg: #dbeafe;
  --gray-50: #f8fafc; --gray-100: #f1f5f9; --gray-200: #e2e8f0; --gray-400: #94a3b8;
  --gray-500: #64748b; --gray-700: #334155; --gray-900: #0f172a;
  --radius: 10px; --shadow: 0 2px 12px rgba(0,0,0,.06);
}
* { margin:0; padding:0; box-sizing:border-box; }
body { font-family:'Segoe UI',system-ui,-apple-system,sans-serif; background:var(--gray-100); color:var(--gray-900); }
.header { background:linear-gradient(135deg,var(--primary),#16213e); color:#fff; padding:16px 28px; display:flex; justify-content:space-between; }
.header h1 { font-size:1.25rem; font-weight:600; }
.header .right { display:flex; align-items:center; gap:14px; }
.header .user-info { font-size:.85rem; opacity:.85; }
.hbtn { background:rgba(255,255,255,.12); border:1px solid rgba(255,255,255,.25); color:#fff; padding:5px 14px; border-radius:10px; }
.hbtn:hover { background:rgba(255,255,255,.22); }
.nav { background:#fff; border-bottom:1px solid var(--gray-200); padding:0 28px; display:flex; gap:0; }
.nav-tab { padding:12px 20px; font-size:.9rem; cursor:pointer; border-bottom:3px solid transparent; color:var(--gray-500); }
.nav-tab:hover { color:var(--primary); background:var(--gray-50); }
.nav-tab.active { color:var(--primary); border-bottom-color:var(--primary); font-weight:600; }
.container { max-width:1240px; margin:auto; padding:20px 24px; }
.page { display:none; }
.page.active { display:block; }
.stats { display:grid; grid-template-columns:repeat(auto-fit,minmax(170px,1fr)); gap:14px; margin-bottom:24px; }
.card { background:#fff; border-radius:var(--radius); padding:20px; box-shadow:var(--shadow); }
.card .lbl { font-size:.78rem; color:var(--gray-500); text-transform:uppercase; letter-spacing:.4px; margin-bottom:4px; }
.card .val { font-size:1.8rem; font-weight:700; color:var(--primary); }
.card.green .val { color:var(--green); }
.card.amber .val { color:var(--amber); }
.card.red .val { color:var(--red); }
.section { background:#fff; border-radius:var(--radius); padding:20px 24px; margin-bottom:18px; box-shadow:var(--shadow); }
.section-head { display:flex; justify-content:space-between; align-items:center; margin-bottom:14px; flex-wrap:wrap; gap:10px; }
.section-head h3 { font-size:1.05rem; color:var(--primary); }
.search-box { padding:7px 14px; border:1px solid var(--gray-200); border-radius:8px; font-size:.88rem; width:260px; outline:none; }
.search-box:focus { border-color:var(--primary); box-shadow:0 0 2px var(--primary-bg); }
table { width:100%; border-collapse:collapse; }
th { text-align:left; padding:9px 12px; background:var(--gray-50); color:var(--gray-500); font-size:.76rem; text-transform:uppercase; }
td { padding:9px 12px; border-bottom:1px solid var(--gray-100); font-size:.88rem; }
tr:hover td { background:var(--gray-50); }
.clickable { color:var(--blue); cursor:pointer; font-weight:500; }
.clickable:hover { text-decoration:underline; }
.badge { display:inline-block; padding:2px 10px; border-radius:10px; font-size:.76rem; font-weight:600; }
.badge.scheduled { background:var(--blue-bg); color:var(--blue); }
.badge.completed { background:var(--green-bg); color:var(--green); }
.badge.cancelled { background:var(--red-bg); color:var(--red); }
.badge.admin { background:var(--red-bg); color:var(--red); }
.badge.doctor { background:var(--green-bg); color:var(--green); }
.badge.receptionist { background:var(--blue-bg); color:var(--blue); }
.empty { text-align:center; padding:30px; color:var(--gray-400); }
.bar { display:flex; flex-direction:column; gap:8px; }
.bar-row { display:flex; align-items:center; gap:10px; }
.bar-lbl { min-width:100px; font-size:.85rem; text-align:right; color:var(--gray-700); }
.bar-track { flex:1; background:var(--gray-200); border-radius:6px; height:26px; overflow:hidden; }
.bar-fill { height:100%; border-radius:6px; display:flex; align-items:center; padding-left:10px; color:#fff; font-size:.78rem; }
.grid-2 { display:grid; grid-template-columns:1fr 1fr; gap:18px; margin-bottom:18px; }
.detail-panel { background:#fff; border-radius:var(--radius); padding:28px; box-shadow:var(--shadow); }
.detail-panel h2 { color:var(--primary); margin-bottom:18px; font-size:1.3rem; }
.detail-grid { display:grid; grid-template-columns:1fr 1fr; gap:12px 24px; margin-bottom:20px; }
.detail-field .dfk { font-size:.76rem; color:var(--gray-500); text-transform:uppercase; letter-spacing:.3px; }
.detail-field .dfv { font-size:1rem; font-weight:500; margin-top:2px; }
.back-btn { display:inline-flex; align-items:center; gap:6px; color:var(--primary); cursor:pointer; font-size:.9rem; font-weight:500; }
.back-btn:hover { text-decoration:underline; }
.login-wrap { display:flex; justify-content:center; align-items:center; min-height:90vh; }
.login-box { background:#fff; padding:36px; border-radius:12px; box-shadow:0 4px 24px rgba(0,0,0,.08); width:340px; }
.login-box h2 { margin-bottom:20px; text-align:center; color:var(--primary); font-size:1.3rem; }
.login-box input { width:100%; padding:10px 14px; margin-bottom:12px; border:1px solid var(--gray-200); border-radius:8px; }
.login-box button { width:100%; padding:10px; background:var(--primary); color:#fff; border:none; border-radius:8px; font-size:1.1rem; font-weight:600; }
.login-box button:hover { background:var(--primary-light); }
.login-box .err { color:var(--red); font-size:.83rem; margin-bottom:8px; text-align:center; }
.pagination { display:flex; justify-content:center; align-items:center; gap:8px; margin-top:14px; padding-top:10px; }
.pg-btn { padding:6px 12px; border:1px solid var(--gray-200); border-radius:6px; font-size:.84rem; cursor:pointer; background:var(--gray-50); }
.pg-btn:hover { background:var(--gray-100); }
.pg-btn.active { background:var(--primary); color:#fff; border-color:var(--primary); }
.pg-btn.disabled { opacity:.4; cursor:default; }
.pg-info { font-size:.84rem; color:var(--gray-500); }
/* Buttons */

```

```

.btn { padding:7px 16px; border:none; border-radius:6px; font-size:.85rem; cursor:pointer; font-weight:500; transition:backg
.btn-primary { background:var(--primary); color:#fff; }
.btn-primary:hover { background:var(--primary-light); }
.btn-success { background:var(--green); color:#fff; }
.btn-success:hover { background:#047857; }
.btn-danger { background:var(--red); color:#fff; }
.btn-danger:hover { background:#b91c1c; }
.btn-sm { padding:4px 10px; font-size:.78rem; }
/* Modal */
.modal-overlay { display:none; position:fixed; top:0; left:0; width:100%; height:100%; background:rgba(0,0,0,.45); z-index:2
.modal-overlay.show { display:flex; }
.modal { background:#fff; border-radius:12px; padding:28px; width:420px; max-width:90vw; max-height:85vh; overflow-y:auto; b
.modal h3 { color:var(--primary); margin-bottom:16px; font-size:1.1rem; }
.modal label { display:block; font-size:.82rem; color:var(--gray-700); margin-bottom:4px; font-weight:500; }
.modal input, .modal select, .modal textarea { width:100%; padding:8px 12px; border:1px solid var(--gray-200); border-radius:6
.modal input:focus, .modal select:focus, .modal textarea:focus { border-color:var(--primary); box-shadow:0 0 2px var(--prima
.modal textarea { resize:vertical; min-height:60px; }
.modal .btn-row { display:flex; justify-content:flex-end; gap:10px; margin-top:8px; }
.modal .err { color:var(--red); font-size:.83rem; margin-bottom:8px; }
.toast { position:fixed; top:20px; right:20px; padding:12px 20px; border-radius:8px; color:#fff; font-size:.88rem; z-index:3
.toast.show { opacity:1; }
.toast.success { background:var(--green); }
.toast.error { background:var(--red); }
.action-btns { display:flex; gap:6px; }
@media(max-width:768px) { .grid-2{grid-template-columns:1fr;} .detail-grid{grid-template-columns:1fr;} .search-box{width:100
</style>
</head>
<body>

<!-- TOAST -->
<div id="toast" class="toast"></div>

<!-- LOGIN -->
<div id="login-screen">
<div class="login-wrap"><div class="login-box">
  <h2>&#9764; Healthcare DMS</h2>
  <div id="login-error" class="err" style="display:none"></div>
  <input id="username" placeholder="Username" value="admin">
  <input id="password" type="password" placeholder="Password" value="admin123">
  <button onclick="doLogin()">Sign In</button>
  <p style="margin-top:14px;font-size:.8rem;color:var(--gray-500);text-align:center">Demo: admin / admin123</p>
</div></div>
</div>

<!-- APP -->
<div id="app" style="display:none">
<div class="header">
  <h1>
    <a href="#" onclick="activateTab('overview'); return false;" style="color: inherit; text-decoration: none;">
      &#9764; Healthcare DMS
    </a>
  </h1>
  <div class="right">
    <span class="user-info" id="user-label"></span>
    <button class="hbtn" onclick="logout()">Logout</button>
  </div>
</div>
<div class="nav" id="nav">
  <div class="nav-tab active" data-page="overview">Overview</div>
  <div class="nav-tab" data-page="patients">Patients</div>
  <div class="nav-tab" data-page="doctors">Doctors</div>
  <div class="nav-tab" data-page="appointments">Appointments</div>
  <div class="nav-tab" data-page="users">Users</div>
</div>
<div class="container">

<!-- OVERVIEW -->
<div id="pg-overview" class="page active">
  <div class="stats" id="stats-cards"><div class="empty">Loading...</div></div>
  <div class="grid-2">
    <div class="section"><h3 style="margin-bottom:12px">Patients by Department</h3><div id="dept-chart" class="bars"></div>
    <div class="section"><h3 style="margin-bottom:12px">Appointment Status</h3><div id="status-chart" class="bars"></div>
  </div>
  <div class="section">
    <div class="section-head"><h3>Recent Patients</h3></div>
    <table><thead><tr><th>Name</th><th>Age</th><th>Gender</th><th>Blood</th><th>Contact</th></tr></thead>
    <tbody id="recent-patients"></tbody></table>
  </div>
</div>

<!-- PATIENTS -->
<div id="pg-patients" class="page">
<div id="patients-list-view">
  <div class="section">
    <div class="section-head">
      <h3 id="patients-count-label">All Patients</h3>
      <div style="display:flex;gap:10px;flex-wrap:wrap;align-items:center">
        <input class="search-box" id="patient-search" placeholder="Search name, contact, address..." oninput="de
        <button class="btn btn-success" onclick="openModal('patient-modal')">+ Add Patient</button>
      </div>
    </div>
    <div>
      <table><thead><tr><th>ID</th><th>Name</th><th>Age</th><th>Gender</th><th>Contact</th><th>Blood</th><th>Actions</th></tr></thead>
      <tbody id="patients-tbody"></tbody></table>
      <div id="patients-pagination" class="pagination"></div>
    </div>
  </div>
<div id="patients-detail-view" style="display:none">

```

```

<div class="detail-panel">
  <div class="back-btn" onclick="showPatientsList()">&#8592; Back to Patients</div>
  <div style="display:flex;justify-content:space-between;align-items:center;flex-wrap:wrap">
    <h2 id="pd-name"></h2>
    <div class="action-btns">
      <button class="btn btn-primary btn-sm" id="pd-edit-btn">Edit</button>
      <button class="btn btn-danger btn-sm" id="pd-delete-btn">Delete</button>
    </div>
  </div>
  <div class="detail-grid" id="pd-fields"></div>
  <h3 style="margin:18px 0 12px;color:var(--primary)">Medical Records</h3>
  <table><thead><tr><th>Date</th><th>Doctor</th><th>Diagnosis</th><th>Prescription</th><th>Notes</th></tr></thead>
  <tbody id="pd-records"></tbody></table>
  <h3 style="margin:18px 0 12px;color:var(--primary)">Appointments</h3>
  <table><thead><tr><th>Date & Time</th><th>Doctor</th><th>Reason</th><th>Status</th></tr></thead>
  <tbody id="pd-appointments"></tbody></table>
</div>
</div>
<!-- DOCTORS -->
<div id="pg-doctors" class="page">
  <div id="doctors-list-view">
    <div class="section">
      <div class="section-head">
        <h3 id="doctors-count-label">All Doctors</h3>
        <div style="display:flex;gap:10px;flex-wrap:wrap;align-items:center">
          <input class="search-box" id="doctor-search" placeholder="Search name, specialization..." oninput="debou
          <select class="search-box" id="dept-filter" onchange="renderDoctors()" style="width:170px"><option value
          <button class="btn btn-success" onclick="openModal('doctor-modal')">+ Add Doctor</button>
        </div>
      </div>
      <table><thead><tr><th>ID</th><th>Name</th><th>Specialization</th><th>Department</th><th>Contact</th><th>Actions</th></tr>
      <tbody id="doctors-tbody"></tbody></table>
    </div>
  </div>
  <div id="doctors-detail-view" style="display:none">
    <div class="detail-panel">
      <div class="back-btn" onclick="showDoctorsList()">&#8592; Back to Doctors</div>
      <div style="display:flex;justify-content:space-between;align-items:center;flex-wrap:wrap">
        <h2 id="dd-name"></h2>
        <div class="action-btns">
          <button class="btn btn-primary btn-sm" id="dd-edit-btn">Edit</button>
          <button class="btn btn-danger btn-sm" id="dd-delete-btn">Delete</button>
        </div>
      </div>
      <div class="detail-grid" id="dd-fields"></div>
      <h3 style="margin:18px 0 12px;color:var(--primary)">Appointments</h3>
      <table><thead><tr><th>Date & Time</th><th>Patient</th><th>Reason</th><th>Status</th></tr></thead>
      <tbody id="dd-appointments"></tbody></table>
    </div>
  </div>
</div>
<!-- APPOINTMENTS -->
<div id="pg-appointments" class="page">
  <div class="section">
    <div class="section-head">
      <h3 id="appts-count-label">All Appointments</h3>
      <div style="display:flex;gap:10px;flex-wrap:wrap;align-items:center">
        <input class="search-box" id="appt-search" placeholder="Search patient, doctor, reason..." oninput="debounce
        <select class="search-box" id="appt-status-filter" onchange="filterAppointments()" style="width:150px">
          <option value="">All Statuses</option><option value="scheduled">Scheduled</option><option value="complet
        </select>
        <button class="btn btn-success" onclick="openAddApptModal()">+ Add Appointment</button>
      </div>
    </div>
    <table><thead><tr><th>ID</th><th>Patient</th><th>Doctor</th><th>Date & Time</th><th>Reason</th><th>Status</th></tr></thead>
    <tbody id="appts-tbody"></tbody></table>
    <div id="appts-pagination" class="pagination"></div>
  </div>
</div>
<!-- USERS -->
<div id="pg-users" class="page">
  <div class="section">
    <div class="section-head">
      <h3 id="users-count-label">All Users</h3>
      <button class="btn btn-success" onclick="openModal('user-modal')">+ Add User</button>
    </div>
    <table><thead><tr><th>ID</th><th>Username</th><th>Full Name</th><th>Role</th></tr></thead>
    <tbody id="users-tbody"></tbody></table>
  </div>
</div>
</div></div>
<!-- ===== MODALS ===== -->
<!-- Add/Edit Patient Modal -->
<div id="patient-modal" class="modal-overlay">
  <div class="modal">
    <h3 id="patient-modal-title">Add New Patient</h3>
    <div id="patient-modal-err" class="err" style="display:none"></div>
    <input type="hidden" id="pm-id">
    <label>Name *</label><input id="pm-name" placeholder="Full name">
  </div>
</div>

```

```

        <label>Age *</label><input id="pm-age" type="number" placeholder="Age" min="0" max="150">
        <label>Gender *</label><select id="pm-gender"><option value="Male">Male</option><option value="Female">Female</option></select>
        <label>Contact *</label><input id="pm-contact" placeholder="Phone number">
        <label>Address</label><input id="pm-address" placeholder="Address">
        <label>Blood Group</label><select id="pm-blood"><option value="">Select</option><option value="A">A</option><option value="B">B</option><option value="AB">AB</option><option value="O">O</option></select>
        <div class="btn-row">
            <button class="btn" style="background:var(--gray-200)" onclick="closeModal('patient-modal')">Cancel</button>
            <button class="btn btn-primary" onclick="savePatient()">Save</button>
        </div>
    </div></div>

<!-- Add/Edit Doctor Modal -->
<div id="doctor-modal" class="modal-overlay">
<div class="modal">
    <h3 id="doctor-modal-title">Add New Doctor</h3>
    <div id="doctor-modal-err" class="err" style="display:none"></div>
    <input type="hidden" id="dm-id">
    <label>Name *</label><input id="dm-name" placeholder="Dr. Full Name">
    <label>Specialization *</label><input id="dm-spec" placeholder="e.g. Cardiologist">
    <label>Department *</label><input id="dm-dept" placeholder="e.g. Cardiology">
    <label>Contact *</label><input id="dm-contact" placeholder="Phone number">
    <div class="btn-row">
        <button class="btn" style="background:var(--gray-200)" onclick="closeModal('doctor-modal')">Cancel</button>
        <button class="btn btn-primary" onclick="saveDoctor()">Save</button>
    </div>
</div></div>

<!-- Add User Modal -->
<div id="user-modal" class="modal-overlay">
<div class="modal">
    <h3>Add New User</h3>
    <div id="user-modal-err" class="err" style="display:none"></div>
    <label>Username *</label><input id="um-username" placeholder="Username">
    <label>Password *</label><input id="um-password" type="password" placeholder="Password">
    <label>Full Name *</label><input id="um-fullname" placeholder="Full name">
    <label>Role *</label><select id="um-role"><option value="receptionist">Receptionist</option><option value="doctor">Doctor</option></select>
    <div class="btn-row">
        <button class="btn" style="background:var(--gray-200)" onclick="closeModal('user-modal')">Cancel</button>
        <button class="btn btn-primary" onclick="saveUser()">Save</button>
    </div>
</div></div>

<!-- Add Appointment Modal -->
<div id="appt-modal" class="modal-overlay">
<div class="modal">
    <h3>Add New Appointment</h3>
    <div id="appt-modal-err" class="err" style="display:none"></div>
    <label>Patient *</label><select id="am-patient"></select>
    <label>Doctor *</label><select id="am-doctor"></select>
    <label>Date & Time *</label><input id="am-datetime" type="datetime-local">
    <label>Reason</label><textarea id="am-reason" placeholder="Reason for visit"></textarea>
    <div class="btn-row">
        <button class="btn" style="background:var(--gray-200)" onclick="closeModal('appt-modal')">Cancel</button>
        <button class="btn btn-primary" onclick="saveAppointment()">Save</button>
    </div>
</div></div>

<!-- Status Change Modal -->
<div id="status-modal" class="modal-overlay">
<div class="modal">
    <h3>Update Appointment Status</h3>
    <input type="hidden" id="sm-id">
    <label>Status</label>
    <select id="sm-status"><option value="scheduled">Scheduled</option><option value="completed">Completed</option><option value="cancelled">Cancelled</option></select>
    <div class="btn-row">
        <button class="btn" style="background:var(--gray-200)" onclick="closeModal('status-modal')">Cancel</button>
        <button class="btn btn-primary" onclick="changeApptStatus()">Update</button>
    </div>
</div></div>

<script>
/* ===== State ===== */
const API='';
let TOKEN='',ROLE='',CURRENT_USER='';
let allPatients=[],allDoctors=[],allAppointments=[],allUsers=[];
const PAGE_SIZE=15;
let patientPage=1,apptPage=1;

/* ===== Helpers ===== */
const _e=document.createElement('div');
function esc(s){if(s==null)return '';_e.textContent=String(s);return _e.innerHTML;}
function debounce(fn,ms){let t;return(...a)=>{clearTimeout(t);t=setTimeout(()=>fn(...a),ms);}}
const debouncedSearchPatients=debounce(()=>{patientPage=1;renderPatients();},250);
const debouncedSearchDoctors=debounce(renderDoctors,250);
const debouncedFilterAppts=debounce(()=>{apptPage=1;renderAppointments();},250);

function toast(msg,type){const t=document.getElementById('toast');t.textContent=msg;t.className='toast '+type+' show';setTime
function hdr(){return{'Authorization':'Bearer '+TOKEN,'Content-Type':'application/json'}};
async function api(path){const r=await fetch(API+path,{headers:hdr()});if(!r.ok)throw new Error(r.status);return r.json();}
async function apiPost(path,body){const r=await fetch(API+path,{method:'POST',headers:hdr(),body:JSON.stringify(body)});if(!r.ok)throw new Error(r.status);return r.json();}
async function apiPut(path,body){const r=await fetch(API+path,{method:'PUT',headers:hdr(),body:JSON.stringify(body)});if(!r.ok)throw new Error(r.status);return r.json();}
async function apiPatch(path,body){const r=await fetch(API+path,{method:'PATCH',headers:hdr(),body:JSON.stringify(body)});if(!r.ok)throw new Error(r.status);return r.json();}
async function apiDelete(path){const r=await fetch(API+path,{method:'DELETE',headers:hdr()});if(!r.ok){const e=await r.json();throw new Error(e.message);}}

function openModal(id){document.getElementById(id).classList.add('show');}
function closeModal(id){document.getElementById(id).classList.remove('show');}
function doctorName(id){const d=allDoctors.find(x=>x.id===id);return d?d.name:'Doctor #'+id;}

```



```

function patientName(id){const p=allPatients.find(x=>x.id===id);return p?p.name:'Patient #'+id;}
function fmtDate(s){if(!s)return'-';const d=new Date(s);return d.toLocaleDateString('en-IN',{day:'2-digit',month:'short',year:'2-digit'});}
function detailFields(obj){return Object.entries(obj).map(([k,v])=><div class="detail-field"><div class="dfk">'+esc(k)+'</div>'+esc(v)+'</div>');}

/* ===== Auth ===== */
async function doLogin(){
  const u=document.getElementById('username').value,p=document.getElementById('password').value;
  const err=document.getElementById('login-error');err.style.display='none';
  try{
    const r=await fetch(API+'/auth/login',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify({username:u,password:p})});
    if(!r.ok){err.textContent='Invalid credentials';err.style.display='block';return;}
    const data = await r.json();
    TOKEN = data.access_token;
    localStorage.setItem('hc_token', TOKEN);

    document.getElementById('login-screen').style.display='none';
    document.getElementById('app').style.display='block';
    initApp();
  }catch(e){err.textContent='Server unreachable';err.style.display='block';}}

function logout(){
  TOKEN='';
  ROLE='';
  localStorage.removeItem('hc_token');
  document.getElementById('app').style.display='none';
  document.getElementById('login-screen').style.display='block';
}

document.getElementById('password').addEventListener('keydown',e=>{if(e.key==='Enter')doLogin();});

/* ===== Nav ===== */
document.querySelectorAll('.nav-tab').forEach(tab=>{
  tab.addEventListener('click',()=>{
    const pageName = tab.dataset.page;
    localStorage.setItem('hc_current_page', pageName);

    document.querySelectorAll('.nav-tab').forEach(t=>t.classList.remove('active'));
    document.querySelectorAll('.page').forEach(p=>p.classList.remove('active'));
    tab.classList.add('active');
    document.getElementById('pg-'+pageName).classList.add('active');

    if(pageName==='patients') renderPatients();
    if(pageName==='doctors') renderDoctors();
    if(pageName==='appointments') renderAppointments();
    if(pageName==='users') renderUsers();
    if(pageName==='overview') loadOverview();
  });
});

function activateTab(name){
  document.querySelectorAll('.nav-tab').forEach(t=>t.classList.remove('active'));
  document.querySelectorAll('.page').forEach(p=>p.classList.remove('active'));
  const tab=document.querySelector('.nav-tab[data-page="'+name+'"]');
  if(tab)tab.classList.add('active');
  document.getElementById('pg-'+name).classList.add('active');
}

/* ===== Init ===== */
async function initApp(){
  try {
    const me = await api('/auth/me');
    ROLE = me.role;
    CURRENT_USER = me.username;
    document.getElementById('user-label').textContent = me.full_name + ' (' + me.role + ')';
  } catch(e) {
    // If the token is expired or invalid, force a logout
    logout();
    return;
  }

  // Show/hide admin-only features
  const isAdmin = ROLE === 'admin';
  document.querySelectorAll('.admin-only').forEach(el => el.style.display = isAdmin ? '' : 'none');
  const usersTab = document.querySelector('.nav-tab[data-page="users"]');
  if(usersTab) usersTab.style.display = isAdmin ? '' : 'none';

  await Promise.all([loadPatients(), loadDoctors(), loadAppointments(), loadUsers()]);
  const savedPage = localStorage.getItem('hc_current_page') || 'overview';

  activateTab(savedPage);

  if (savedPage === 'overview') loadOverview();
  else if (savedPage === 'patients') renderPatients();
  else if (savedPage === 'doctors') renderDoctors();
  else if (savedPage === 'appointments') renderAppointments();
  else if (savedPage === 'users') renderUsers();
}

async function loadPatients(){try{allPatients=await api('/patients/');}catch(e){allPatients=[];}}
async function loadDoctors(){try{allDoctors=await api('/doctors/');}catch(e){allDoctors=[];}} populateDeptFilter();
async function loadAppointments(){try{allAppointments=await api('/appointments/');}catch(e){allAppointments=[];}}
async function loadUsers(){try{allUsers=await api('/auth/users');}catch(e){allUsers=[];}}

/* ===== Overview ===== */
async function loadOverview(){
  try{
    const s=await api('/analytics/summary');
    document.getElementById('stats-cards').innerHTML=
      '<div class="card"><div class="lbl">Patients</div><div class="val">'+s.total_patients+'</div></div>'+
      '<div class="card"><div class="lbl">Doctors</div><div class="val">'+s.total_doctors+'</div></div>'+

```

```

        '<div class="card"><div class="lbl">Appointments</div><div class="val">'+s.total_appointments+'</div></div>'+
        '<div class="card green"><div class="lbl">Completed</div><div class="val">'+(s.appointments_by_status.completed|
        '<div class="card amber"><div class="lbl">Scheduled</div><div class="val">'+(s.appointments_by_status.scheduled|
        '<div class="card red"><div class="lbl">Cancelled</div><div class="val">'+(s.appointments_by_status.cancelled|
        const depts=s.patients_by_department||[],maxD=Math.max(...Object.values(depts),1);
        document.getElementById('dept-chart').innerHTML=Object.entries(depts).sort((a,b)=>b[1]-a[1]).map(([d,c])=>'<div clas
        const st=s.appointments_by_status||[],maxS=Math.max(st.scheduled||0,st.completed||0,st.cancelled||0,1);
        document.getElementById('status-chart').innerHTML=[{l:'Scheduled',v:st.scheduled||0,c:'var(--blue)'},{l:'Completed',
        }catch(e){document.getElementById('stats-cards').innerHTML='<div class="empty">Analytics requires admin role</div>';}
        const recent=allPatients.slice(0,10);
        document.getElementById('recent-patients').innerHTML=recent.map(p=>'<tr><td><span class="clickable" onclick="showPatient
    }
    /* ===== Patients ===== */
    function renderPatients(){
        const q=(document.getElementById('patient-search').value|| '').toLowerCase();
        let f=allPatients;
        if(q)f=f.filter(p=>p.name.toLowerCase().includes(q)||p.contact.includes(q)||p.address.toLowerCase().includes(q)||
        const total=f.length,pages=Math.ceil(total/PAGE_SIZE)||1;
        if(patientPage>pages)patientPage=pages;
        const start=(patientPage-1)*PAGE_SIZE,slice=f.slice(start,start+PAGE_SIZE);
        document.getElementById('patients-count-label').textContent='All Patients ('+total+')';
        document.getElementById('patients-tbody').innerHTML=slice.map(p=>
            '<tr><td>'+p.id+'</td><td><span class="clickable" onclick="showPatientDetail('+p.id+')">'+esc(p.name)+'</span></td><td>
        ).join('')|'|<tr><td colspan="7" class="empty">No patients found</td></tr>';
        renderPagination('patients-pagination',patientPage,pages,p=>{patientPage=p;renderPatients();},total);
    }
    async function showPatientDetail(id){
        const p=allPatients.find(x=>x.id===id);if(!p)return;
        document.getElementById('patients-list-view').style.display='none';
        document.getElementById('patients-detail-view').style.display='block';
        document.getElementById('pd-name').textContent=p.name;
        document.getElementById('pd-fields').innerHTML=detailFields({'Patient ID':p.id,Age:p.age,Gender:p.gender,'Blood Group':p
        document.getElementById('pd-edit-btn').onclick=(=>openEditPatient(id);
        document.getElementById('pd-delete-btn').onclick=(=>deletePatient(id);showPatientsList(););
        activateTab('patients');
        try{
            const recs=await api('/patients/'+id+'/records');
            document.getElementById('pd-records').innerHTML=recs.map(r=>'<tr><td>'+fmtDate(r.created_at)+'</td><td><span class="
        }catch(e){document.getElementById('pd-records').innerHTML='<tr><td colspan="5" class="empty">Access denied or error</td>
        const appts=allAppointments.filter(a=>a.patient_id===id);
        document.getElementById('pd-appointments').innerHTML=appts.map(a=>'<tr><td>'+fmtDate(a.date_time)+'</td><td><span class="
    }
    function showPatientsList(){document.getElementById('patients-detail-view').style.display='none';document.getElementById('pa
    function openEditPatient(id){
        const p=allPatients.find(x=>x.id===id);if(!p)return;
        document.getElementById('patient-modal-title').textContent='Edit Patient';
        document.getElementById('pm-id').value=p.id;
        document.getElementById('pm-name').value=p.name;
        document.getElementById('pm-age').value=p.age;
        document.getElementById('pm-gender').value=p.gender;
        document.getElementById('pm-contact').value=p.contact;
        document.getElementById('pm-address').value=p.address|| '';
        document.getElementById('pm-blood').value=p.blood_group|| '';
        document.getElementById('patient-modal-err').style.display='none';
        openModal('patient-modal');
    }
    async function savePatient(){
        const err=document.getElementById('patient-modal-err');err.style.display='none';
        const id=document.getElementById('pm-id').value;
        const name=document.getElementById('pm-name').value.trim();
        const age=parseInt(document.getElementById('pm-age').value);
        const gender=document.getElementById('pm-gender').value;
        const contact=document.getElementById('pm-contact').value.trim();
        const address=document.getElementById('pm-address').value.trim();
        const blood_group=document.getElementById('pm-blood').value;
        if(!name||!age||!contact){err.textContent='Name, age, and contact are required';err.style.display='block';return;}
        try{
            if(id){
                await apiPut('/patients/'+id,{name,age,gender,contact,address,blood_group});
                toast('Patient updated','success');
            }else{
                await apiPost('/patients/',{name,age,gender,contact,address,blood_group});
                toast('Patient created','success');
            }
            closeModal('patient-modal');
            await loadPatients();renderPatients();loadOverview();
        }catch(e){err.textContent=e.message;err.style.display='block';}
    }
    async function deletePatient(id){
        if(!confirm('Delete this patient?'))return;
        try{await apiDelete('/patients/'+id);toast('Patient deleted','success');await loadPatients();renderPatients();loadOverview
        }catch(e){toast('Error: '+e.message,'error');}
    }
    /* ===== Doctors ===== */
    function populateDeptFilter(){
        const depts=[...new Set(allDoctors.map(d=>d.department))].sort();
        document.getElementById('dept-filter').innerHTML='<option value="">All Departments ('+depts.length+')</option>'+depts.ma
    }
    function renderDoctors(){
        const q=(document.getElementById('doctor-search').value|| '').toLowerCase();
        const dept=document.getElementById('dept-filter').value;
        let f=allDoctors;
        if(q)f=f.filter(d=>d.name.toLowerCase().includes(q)||d.specialization.toLowerCase().includes(q)||d.department.toLowerCase

```

```

        if(dept)f=f.filter(d=>d.department===dept);
        document.getElementById('doctors-count-label').textContent='All Doctors ('+f.length+')';
        document.getElementById('doctors-tbody').innerHTML=f.map(d=>
            '<tr><td>'+d.id+'</td><td><span class="clickable" onclick="showDoctorDetail('+d.id+')">'+esc(d.name)+'</span></td><td>'
        ).join('')|'|<tr><td colspan="6" class="empty">No doctors found</td></tr>';
    }
    async function showDoctorDetail(id){
        const d=allDoctors.find(x=>x.id===id);if(!d)return;
        document.getElementById('doctors-list-view').style.display='none';
        document.getElementById('doctors-detail-view').style.display='block';
        document.getElementById('dd-name').textContent=d.name;
        document.getElementById('dd-fields').innerHTML=detailFields({Doctor ID:d.id,Specialization:d.specialization,Department:d.department});
        document.getElementById('dd-edit-btn').onclick={()=>openEditDoctor(id);
        document.getElementById('dd-delete-btn').onclick={()=>deleteDoctor(id);showDoctorsList();};
        activateTab('doctors');
        const appts=allAppointments.filter(a=>a.doctor_id===id);
        document.getElementById('dd-appointments').innerHTML=appts.map(a=>'<tr><td>'+fmtDate(a.date_time)+'</td><td><span class="clickable" onclick="showPatientDetail('+a.patient_id+')">'+esc(patientName(a.patient_id))+</span></td><td>'
    }
    function showDoctorsList(){document.getElementById('doctors-detail-view').style.display='none';document.getElementById('doctors-list-view').style.display='block';}

    function openEditDoctor(id){
        const d=allDoctors.find(x=>x.id===id);if(!d)return;
        document.getElementById('doctor-modal-title').textContent='Edit Doctor';
        document.getElementById('dm-id').value=d.id;
        document.getElementById('dm-name').value=d.name;
        document.getElementById('dm-spec').value=d.specialization;
        document.getElementById('dm-dept').value=d.department;
        document.getElementById('dm-contact').value=d.contact;
        document.getElementById('doctor-modal-err').style.display='none';
        openModal('doctor-modal');
    }

    async function saveDoctor(){
        const err=document.getElementById('doctor-modal-err');err.style.display='none';
        const id=document.getElementById('dm-id').value;
        const name=document.getElementById('dm-name').value.trim();
        const specialization=document.getElementById('dm-spec').value.trim();
        const department=document.getElementById('dm-dept').value.trim();
        const contact=document.getElementById('dm-contact').value.trim();
        if(!name||!specialization||!department||!contact){err.textContent='All fields are required';err.style.display='block';return;}
        try{
            if(id){
                await apiPut('/doctors/'+id,{name,specialization,department,contact});
                toast('Doctor updated','success');
            }else{
                await apiPost('/doctors/',{name,specialization,department,contact});
                toast('Doctor created','success');
            }
            closeModal('doctor-modal');
            await loadDoctors();renderDoctors();loadOverview();
        }catch(e){err.textContent=e.message;err.style.display='block';}
    }

    async function deleteDoctor(id){
        if(!confirm('Delete this doctor?'))return;
        try{await apiDelete('/doctors/'+id);toast('Doctor deleted','success');await loadDoctors();renderDoctors();loadOverview();}
        catch(e){toast('Error: '+e.message,'error');}
    }

    /* ===== Users ===== */
    function renderUsers(){
        document.getElementById('users-count-label').textContent='All Users ('+allUsers.length+')';
        document.getElementById('users-tbody').innerHTML=allUsers.map(u=>
            '<tr><td>'+u.id+'</td><td>'+esc(u.username)+'</td><td>'+esc(u.full_name)+'</td><td><span class="badge '+u.role+'">'+esc(u.role)+'</span></td>'
        ).join('')|'|<tr><td colspan="4" class="empty">No users</td></tr>';
    }

    async function saveUser(){
        const err=document.getElementById('user-modal-err');err.style.display='none';
        const username=document.getElementById('um-username').value.trim();
        const password=document.getElementById('um-password').value;
        const full_name=document.getElementById('um-fullname').value.trim();
        const role=document.getElementById('um-role').value;
        if(!username||!password||!full_name){err.textContent='All fields are required';err.style.display='block';return;}
        try{
            await apiPost('/auth/register',{username,password,full_name,role});
            toast('User created','success');
            closeModal('user-modal');
            document.getElementById('um-username').value='';document.getElementById('um-password').value='';document.getElementById('um-fullname').value='';document.getElementById('um-role').value='';
            await loadUsers();renderUsers();
        }catch(e){err.textContent=e.message;err.style.display='block';}
    }

    /* ===== Appointments ===== */
    function filterAppointments(){apptPage=1;renderAppointments();}
    function renderAppointments(){
        const q=(document.getElementById('appt-search').value||').toLowerCase();
        const st=document.getElementById('appt-status-filter').value;
        let f=allAppointments;
        if(q)f=f.filter(a=>(a.reason||').toLowerCase().includes(q)||patientName(a.patient_id).toLowerCase().includes(q)||doctorName(a.doctor_id).toLowerCase().includes(q));
        if(st)f=f.filter(a=>a.status===st);
        const total=f.length,pages=Math.ceil(total/PAGE_SIZE)||1;
        if(apptPage>pages)apptPage=pages;
        const start=(apptPage-1)*PAGE_SIZE,slice=f.slice(start,start+PAGE_SIZE);
        document.getElementById('appts-count-label').textContent='All Appointments ('+total+')';
        document.getElementById('appts-tbody').innerHTML=slice.map(a=>
            '<tr><td>'+a.id+'</td><td>'
            '<td><span class="clickable" onclick="showPatientDetail('+a.patient_id+')">'+esc(patientName(a.patient_id))+</span>'
            '<td><span class="clickable" onclick="showDoctorDetail('+a.doctor_id+')">'+esc(doctorName(a.doctor_id))+</span></td><td>'
        ).join('')|'|<tr><td colspan="4" class="empty">No appointments found</td></tr>';
    }

```

```

        '<td>'+fmtDate(a.date_time)+'</td><td>'+esc(a.reason)+'</td>'+
        '<td><span class="badge '+a.status+'" style="cursor:pointer" onclick="openStatusModal('+a.id+',\''+a.status+'\')">'+
        '<td><button class="btn btn-danger btn-sm" onclick="deleteAppointment('+a.id+')">Del</button></td></tr>'
    ).join('')||'<tr><td colspan="7" class="empty">No appointments found</td></tr>';
    renderPagination('appts-pagination', apptPage, pages, p=>{apptPage=p;renderAppointments();}, total);
}
function openAddApptModal(){
    document.getElementById('am-patient').innerHTML=allPatients.map(p=>'<option value="'+p.id+'">'+esc(p.name)+' (ID: '+p.id+'>'+
    document.getElementById('am-doctor').innerHTML=allDoctors.map(d=>'<option value="'+d.id+'">'+esc(d.name)+' - '+esc(d.dep
    document.getElementById('am-datetime').value='';document.getElementById('am-reason').value='';
    document.getElementById('appt-modal-err').style.display='none';
    openModal('appt-modal');
}
async function saveAppointment(){
    const err=document.getElementById('appt-modal-err');err.style.display='none';
    const patient_id=parseInt(document.getElementById('am-patient').value);
    const doctor_id=parseInt(document.getElementById('am-doctor').value);
    const dt=document.getElementById('am-datetime').value;
    const reason=document.getElementById('am-reason').value.trim();
    if(!patient_id||!doctor_id||!dt){err.textContent='Patient, doctor, and date are required';err.style.display='block';return}
    try{
        await apiPost('/appointments/',{patient_id,doctor_id,date_time:new Date(dt).toISOString(),reason});
        toast('Appointment created','success');
        closeModal('appt-modal');
        await loadAppointments();renderAppointments();loadOverview();
    }catch(e){err.textContent=e.message;err.style.display='block';}
}
function openStatusModal(id,current){
    document.getElementById('sm-id').value=id;
    document.getElementById('sm-status').value=current;
    openModal('status-modal');
}
async function changeApptStatus(){
    const id=document.getElementById('sm-id').value;
    const status=document.getElementById('sm-status').value;
    try{
        await apiPatch('/appointments/'+id,{status});
        toast('Status updated','success');closeModal('status-modal');
        await loadAppointments();renderAppointments();loadOverview();
    }catch(e){toast('Error: '+e.message,'error');}
}
async function deleteAppointment(id){
    if(!confirm('Delete this appointment?'))return;
    try{await apiDelete('/appointments/'+id);toast('Appointment deleted','success');await loadAppointments();renderAppointments();}
    catch(e){toast('Error: '+e.message,'error');}
}
/* ===== Pagination ===== */
function renderPagination(eId,current,total,onPage,count){
    const el=document.getElementById(eId);
    if(total<=1){el.innerHTML=count?'<span class="pg-info">'+count+' records</span>':'';return;}
    let h='<span class="pg-info">'+count+' records</span>';
    h+=<button class="pg-btn" id="'+eId+'-prev" '+ (current<=1?'disabled':'')+'>#171; Prev</button>';
    let s=Math.max(1,current-2),e=Math.min(total,s+4);if(e-s<4)s=Math.max(1,e-4);
    for(let i=s;i<=e;i++)h+=<button class="pg-btn" '+ (i===current?'active':'')+' data-pg="'+i+'">'+i+'</button>';
    h+=<button class="pg-btn" id="'+eId+'-next" '+ (current>=total?'disabled':'')+'>Next &#187;</button>';
    el.innerHTML=h;
    el.querySelector('#'+eId+'-prev').addEventListener('click',()=>{if(current>1)onPage(current-1)});
    el.querySelector('#'+eId+'-next').addEventListener('click',()=>{if(current<total)onPage(current+1)});
    el.querySelectorAll('[data-pg]').forEach(b=>b.addEventListener('click',()=>onPage(parseInt(b.dataset.pg))));
}
/* ===== Reset modal fields on open for "Add" ===== */
document.querySelector('#patient-modal .btn-success,button[onclick="openModal(\'patient-modal\')"]');
/* We handle resetting via the + Add buttons: */
document.querySelectorAll('button').forEach(b=>{
    if(b.textContent.includes('+ Add Patient'))b.addEventListener('click',()=>{
        document.getElementById('patient-modal-title').textContent='Add New Patient';
        document.getElementById('pm-id').value='';document.getElementById('pm-name').value='';
        document.getElementById('pm-age').value='';document.getElementById('pm-contact').value='';
        document.getElementById('pm-address').value='';document.getElementById('pm-blood').value='';
        document.getElementById('patient-modal-err').style.display='none';
    });
    if(b.textContent.includes('+ Add Doctor'))b.addEventListener('click',()=>{
        document.getElementById('doctor-modal-title').textContent='Add New Doctor';
        document.getElementById('dm-id').value='';document.getElementById('dm-name').value='';
        document.getElementById('dm-spec').value='';document.getElementById('dm-dept').value='';
        document.getElementById('dm-contact').value='';
        document.getElementById('doctor-modal-err').style.display='none';
    });
    if(b.textContent.includes('+ Add User'))b.addEventListener('click',()=>{
        document.getElementById('um-username').value='';document.getElementById('um-password').value='';
        document.getElementById('um-fullname').value='';
        document.getElementById('user-modal-err').style.display='none';
    });
});
window.addEventListener('DOMContentLoaded', () => {
    const savedToken = localStorage.getItem('hc_token');
    if (savedToken) {
        TOKEN = savedToken;
        document.getElementById('login-screen').style.display = 'none';
        document.getElementById('app').style.display = 'block';
        initApp();
    }
});
}

```

```
</script>  
</body>  
</html>  
"""
```

app/routers/doctors.py

```
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from typing import Optional

from app.database import get_db
from app.models.doctor import Doctor
from app.models.user import User
from app.schemas.doctor import DoctorCreate, DoctorUpdate, DoctorOut
from app.auth import require_roles

router = APIRouter(prefix="/doctors", tags=["Doctors"])

@router.get("/", response_model=list[DoctorOut])
def list_doctors(
    search: Optional[str] = Query(None, description="Search by name, specialization, or department"),
    department: Optional[str] = Query(None, description="Filter by department"),
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin", "doctor", "receptionist")),
):
    query = db.query(Doctor)
    if search:
        query = query.filter(
            Doctor.name.ilike(f"%{search}%")
            | Doctor.specialization.ilike(f"%{search}%")
            | Doctor.department.ilike(f"%{search}%")
        )
    if department:
        query = query.filter(Doctor.department.ilike(f"%{department}%"))
    return query.order_by(Doctor.id.desc()).all()

@router.post("/", response_model=DoctorOut, status_code=201)
def create_doctor(
    body: DoctorCreate,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin")),
):
    doctor = Doctor(**body.model_dump())
    db.add(doctor)
    db.commit()
    db.refresh(doctor)
    return doctor

@router.get("/{doctor_id}", response_model=DoctorOut)
def get_doctor(
    doctor_id: int,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin", "doctor", "receptionist")),
):
    doctor = db.query(Doctor).filter(Doctor.id == doctor_id).first()
    if not doctor:
        raise HTTPException(status_code=404, detail="Doctor not found")
    return doctor

@router.put("/{doctor_id}", response_model=DoctorOut)
def update_doctor(
    doctor_id: int,
    body: DoctorUpdate,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin")),
):
    doctor = db.query(Doctor).filter(Doctor.id == doctor_id).first()
    if not doctor:
        raise HTTPException(status_code=404, detail="Doctor not found")
    for field, value in body.model_dump(exclude_unset=True).items():
        setattr(doctor, field, value)
    db.commit()
    db.refresh(doctor)
    return doctor

@router.delete("/{doctor_id}", status_code=204)
def delete_doctor(
    doctor_id: int,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin")),
):
    doctor = db.query(Doctor).filter(Doctor.id == doctor_id).first()
    if not doctor:
        raise HTTPException(status_code=404, detail="Doctor not found")
    db.delete(doctor)
    db.commit()
```

app/routers/patients.py

```
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from typing import Optional

from app.database import get_db
from app.models.patient import Patient
from app.models.user import User
from app.schemas.patient import PatientCreate, PatientUpdate, PatientOut
from app.auth import require_roles

router = APIRouter(prefix="/patients", tags=["Patients"])

@router.get("/", response_model=list[PatientOut])
def list_patients(
    search: Optional[str] = Query(None, description="Search by name"),
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin", "doctor", "receptionist")),
):
    query = db.query(Patient).filter(Patient.is_active == True)
    if search:
        query = query.filter(Patient.name.ilike(f"%{search}%"))
    return query.order_by(Patient.id.desc()).all()

@router.post("/", response_model=PatientOut, status_code=201)
def create_patient(
    body: PatientCreate,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin", "receptionist")),
):
    patient = Patient(**body.model_dump())
    db.add(patient)
    db.commit()
    db.refresh(patient)
    return patient

@router.get("/{patient_id}", response_model=PatientOut)
def get_patient(
    patient_id: int,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin", "doctor", "receptionist")),
):
    patient = db.query(Patient).filter(Patient.id == patient_id, Patient.is_active == True).first()
    if not patient:
        raise HTTPException(status_code=404, detail="Patient not found")
    return patient

@router.put("/{patient_id}", response_model=PatientOut)
def update_patient(
    patient_id: int,
    body: PatientUpdate,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin", "receptionist")),
):
    patient = db.query(Patient).filter(Patient.id == patient_id, Patient.is_active == True).first()
    if not patient:
        raise HTTPException(status_code=404, detail="Patient not found")
    for field, value in body.model_dump(exclude_unset=True).items():
        setattr(patient, field, value)
    db.commit()
    db.refresh(patient)
    return patient

@router.delete("/{patient_id}", status_code=204)
def delete_patient(
    patient_id: int,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin")),
):
    patient = db.query(Patient).filter(Patient.id == patient_id, Patient.is_active == True).first()
    if not patient:
        raise HTTPException(status_code=404, detail="Patient not found")
    patient.is_active = False # soft delete
    db.commit()
```

app/routers/records.py

```
from pathlib import Path

from fastapi import APIRouter, Depends, HTTPException, UploadFile, File
from sqlalchemy.orm import Session

from app.config import settings
from app.database import get_db
from app.models.medical_record import MedicalRecord
from app.models.patient import Patient
from app.models.doctor import Doctor
from app.models.user import User
from app.schemas.medical_record import MedicalRecordCreate, MedicalRecordOut
from app.auth import require_roles
from app.storage import get_storage

router = APIRouter(tags=["Medical Records"])

@router.get("/patients/{patient_id}/records", response_model=list[MedicalRecordOut])
def get_patient_records(
    patient_id: int,
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("admin", "doctor")),
):
    if not db.query(Patient).filter(Patient.id == patient_id).first():
        raise HTTPException(status_code=404, detail="Patient not found")
    return db.query(MedicalRecord).filter(MedicalRecord.patient_id == patient_id).order_by(MedicalRecord.created_at.desc())

@router.post("/patients/{patient_id}/records", response_model=MedicalRecordOut, status_code=201)
def create_record(
    patient_id: int,
    body: MedicalRecordCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles("doctor")),
):
    if not db.query(Patient).filter(Patient.id == patient_id, Patient.is_active == True).first():
        raise HTTPException(status_code=404, detail="Patient not found")
    # Find the doctor linked to this user
    doctor = db.query(Doctor).filter(Doctor.user_id == current_user.id).first()
    if not doctor:
        raise HTTPException(status_code=400, detail="No doctor profile linked to this user")
    record = MedicalRecord(
        patient_id=patient_id,
        doctor_id=doctor.id,
        diagnosis=body.diagnosis,
        prescription=body.prescription,
        notes=body.notes,
    )
    db.add(record)
    db.commit()
    db.refresh(record)
    return record

@router.post("/upload/{record_id}", status_code=200)
def upload_file(
    record_id: int,
    file: UploadFile = File(...),
    db: Session = Depends(get_db),
    _: User = Depends(require_roles("doctor", "admin")),
):
    record = db.query(MedicalRecord).filter(MedicalRecord.id == record_id).first()
    if not record:
        raise HTTPException(status_code=404, detail="Record not found")

    # Validate extension
    ext = Path(file.filename).suffix.lower()
    if ext not in settings.ALLOWED_EXTENSIONS:
        raise HTTPException(status_code=400, detail=f"File type {ext} not allowed")

    # Validate size
    contents = file.file.read()
    if len(contents) > settings.MAX_UPLOAD_SIZE_MB * 1024 * 1024:
        raise HTTPException(status_code=400, detail="File too large")

    # Save via the configured backend (LocalStorage in Phase 1, S3Storage in Phase 2)
    storage = get_storage()
    key = storage.save(file.filename, contents, content_type=file.content_type or "application/octet-stream")
    record.file_path = key
    db.commit()
    return {"message": "File uploaded", "file_path": key, "backend": settings.STORAGE_BACKEND}
```


app/schemas/__init__.py

app/schemas/appointment.py

```
from pydantic import BaseModel
from datetime import datetime
from typing import Optional

class AppointmentCreate(BaseModel):
    patient_id: int
    doctor_id: int
    date_time: datetime
    reason: str = ""

class AppointmentUpdate(BaseModel):
    status: str # scheduled, completed, cancelled

class AppointmentOut(BaseModel):
    id: int
    patient_id: int
    doctor_id: int
    date_time: datetime
    reason: str
    status: str

    model_config = {"from_attributes": True}
```

app/schemas/doctor.py

```
from pydantic import BaseModel
from typing import Optional

class DoctorCreate(BaseModel):
    name: str
    specialization: str
    department: str
    contact: str
    user_id: Optional[int] = None

class DoctorUpdate(BaseModel):
    name: Optional[str] = None
    specialization: Optional[str] = None
    department: Optional[str] = None
    contact: Optional[str] = None

class DoctorOut(BaseModel):
    id: int
    name: str
    specialization: str
    department: str
    contact: str
    user_id: Optional[int] = None

    model_config = {"from_attributes": True}
```

app/schemas/medical_record.py

```
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class MedicalRecordCreate(BaseModel):
    diagnosis: str
    prescription: str = ""
    notes: str = ""

class MedicalRecordOut(BaseModel):
    id: int
    patient_id: int
    doctor_id: int
    diagnosis: str
    prescription: str
    notes: str
    file_path: str
    created_at: Optional[datetime] = None

    model_config = {"from_attributes": True}
```

app/schemas/patient.py

```
from pydantic import BaseModel
from typing import Optional

class PatientCreate(BaseModel):
    name: str
    age: int
    gender: str
    contact: str
    address: str = ""
    blood_group: str = ""

class PatientUpdate(BaseModel):
    name: Optional[str] = None
    age: Optional[int] = None
    gender: Optional[str] = None
    contact: Optional[str] = None
    address: Optional[str] = None
    blood_group: Optional[str] = None

class PatientOut(BaseModel):
    id: int
    name: str
    age: int
    gender: str
    contact: str
    address: str
    blood_group: str
    is_active: bool

    model_config = {"from_attributes": True}
```

app/schemas/user.py

```
from pydantic import BaseModel

class UserCreate(BaseModel):
    username: str
    password: str
    full_name: str
    role: str = "receptionist"

class UserOut(BaseModel):
    id: int
    username: str
    full_name: str
    role: str

    model_config = {"from_attributes": True}

class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"

class LoginRequest(BaseModel):
    username: str
    password: str
```

app/storage.py

```
"""Pluggable storage backend.
```

```
Phase 1 used the local filesystem (`uploads/`). Phase 2 introduces an S3 backend so the same FastAPI code runs unchanged on AWS EC2.
```

```
The backend is selected at import time via `settings.STORAGE_BACKEND`:
```

```
* `local` – write to `settings.UPLOAD_DIR` (Phase 1 behaviour)
* `s3` – write to `settings.S3_BUCKET` using boto3
```

```
Both backends share the same interface:
```

```
save(filename, content_bytes, content_type) -> str (storage key)
url(key) -> str (download URL)
delete(key) -> None
```

```
This keeps callers (routers/records.py) identical between phases.
```

```
"""
```

```
from __future__ import annotations
```

```
import os
```

```
import uuid
```

```
from pathlib import Path
```

```
from typing import Protocol
```

```
from app.config import settings
```

```
class StorageBackend(Protocol):
```

```
    """Storage backend protocol – both Local and S3 implement this."""
```

```
    def save(self, filename: str, content: bytes, content_type: str = "application/octet-stream") -> str: ...
```

```
    def url(self, key: str) -> str: ...
```

```
    def delete(self, key: str) -> None: ...
```

```
class LocalStorage:
```

```
    """Filesystem-backed storage (Phase 1 behaviour, also used in tests)."""
```

```
    def __init__(self, root: str):
```

```
        self.root = Path(root)
```

```
        self.root.mkdir(parents=True, exist_ok=True)
```

```
    def save(self, filename: str, content: bytes, content_type: str = "application/octet-stream") -> str:
```

```
        ext = Path(filename).suffix.lower()
```

```
        key = f"{uuid.uuid4().hex}.{ext}"
```

```
        (self.root / key).write_bytes(content)
```

```
        return str(self.root / key)
```

```
    def url(self, key: str) -> str:
```

```
        # For local storage the "URL" is just the filesystem path.
```

```
        return key
```

```
    def delete(self, key: str) -> None:
```

```
        p = Path(key)
```

```
        if p.exists():
```

```
            p.unlink()
```

```
class S3Storage:
```

```
    """S3-backed storage (Phase 2)."""
```

```
    def __init__(self, bucket: str, region: str, prefix: str = "medical-records/"):
        # boto3 is imported lazily so local-only environments don't need it
```

```
        import boto3 # type: ignore
```

```
        self.bucket = bucket
```

```
        self.prefix = prefix.rstrip("/") + "/"
```

```
        self.region = region
```

```
        self.client = boto3.client("s3", region_name=region)
```

```
    def save(self, filename: str, content: bytes, content_type: str = "application/octet-stream") -> str:
```

```
        ext = Path(filename).suffix.lower()
```

```
        key = f"{self.prefix}{uuid.uuid4().hex}.{ext}"
```

```
        # Server-side encryption with KMS – required for HIPAA/DISHA compliance
```

```
        extra = {"ContentType": content_type, "ServerSideEncryption": "AES256"}
```

```
        if settings.S3_KMS_KEY_ID:
```

```
            extra["ServerSideEncryption"] = "aws:kms"
```

```
            extra["SSEKMSKeyId"] = settings.S3_KMS_KEY_ID
```

```
        self.client.put_object(Bucket=self.bucket, Key=key, Body=content, **extra)
```

```
        return key
```

```
    def url(self, key: str) -> str:
```

```
        # Pre-signed URL with 1 hour expiry – never expose raw S3 URLs
```

```
        return self.client.generate_presigned_url(
```

```
            "get_object",
```

```
            Params={"Bucket": self.bucket, "Key": key},
```

```
            ExpiresIn=3600,
```

```
        )
```

```
    def delete(self, key: str) -> None:
```

```
        self.client.delete_object(Bucket=self.bucket, Key=key)
```

```
def build_storage() -> StorageBackend:
```

```
    """Factory that returns the configured storage backend."""
```

```
    backend = (getattr(settings, "STORAGE_BACKEND", "local") or "local").lower()
```

```
    if backend == "s3":
```

```
        bucket = settings.S3_BUCKET
```

```
        if not bucket:
```

```
            raise RuntimeError("STORAGE_BACKEND=s3 but S3_BUCKET is not configured")
```

```
        return S3Storage()
```

```

        bucket=bucket,
        region=getattr(settings, "AWS_REGION", os.getenv("AWS_REGION", "us-east-1")),
        prefix=getattr(settings, "S3_PREFIX", "medical-records/"),
    )
    return LocalStorage(settings.UPLOAD_DIR)

# Module-level singleton – but built lazily so tests can monkeypatch settings first.
_storage: StorageBackend | None = None

def get_storage() -> StorageBackend:
    global _storage
    if _storage is None:
        _storage = build_storage()
    return _storage

def reset_storage() -> None:
    """Test helper – drop the cached backend so the next call rebuilds it."""
    global _storage
    _storage = None

```


tests/__init__.py

tests/conftest.py

```
"""Shared pytest fixtures for all tests."""

import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool

from app.database import Base, get_db
from app.main import app
from app.auth import hash_password
from app.models.user import User
from app.models.doctor import Doctor

# In-memory SQLite for tests – isolated from real DB
engine = create_engine("sqlite://", connect_args={"check_same_thread": False}, poolclass=StaticPool)
TestSession = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def override_get_db():
    db = TestSession()
    try:
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db

@pytest.fixture(autouse=True)
def setup_db():
    """Create tables before each test, drop after."""
    Base.metadata.create_all(bind=engine)
    yield
    Base.metadata.drop_all(bind=engine)

@pytest.fixture
def client():
    return TestClient(app)

@pytest.fixture
def db():
    db = TestSession()
    yield db
    db.close()

@pytest.fixture
def admin_token(client, db):
    """Create admin user and return JWT token."""
    user = User(username="admin", password_hash=hash_password("admin123"), full_name="Admin", role="admin")
    db.add(user)
    db.commit()
    resp = client.post("/auth/login", json={"username": "admin", "password": "admin123"})
    return resp.json()["access_token"]

@pytest.fixture
def doctor_token(client, db):
    """Create doctor user + doctor profile and return JWT token."""
    user = User(username="doc1", password_hash=hash_password("doc123"), full_name="Dr. Test", role="doctor")
    db.add(user)
    db.flush()
    doctor = Doctor(user_id=user.id, name="Dr. Test", specialization="General", department="General", contact="1234567890")
    db.add(doctor)
    db.commit()
    resp = client.post("/auth/login", json={"username": "doc1", "password": "doc123"})
    return resp.json()["access_token"]

@pytest.fixture
def receptionist_token(client, db):
    """Create receptionist user and return JWT token."""
    user = User(username="recep1", password_hash=hash_password("recep123"), full_name="Receptionist", role="receptionist")
    db.add(user)
    db.commit()
    resp = client.post("/auth/login", json={"username": "recep1", "password": "recep123"})
    return resp.json()["access_token"]

def auth_header(token: str) -> dict:
    return {"Authorization": f"Bearer {token}"}
```

tests/test_appointments.py

```
from datetime import datetime, timedelta

from app.models.patient import Patient
from app.models.doctor import Doctor
from tests.conftest import auth_header

def _create_patient_and_doctor(db):
    p = Patient(name="P1", age=30, gender="Male", contact="111")
    d = Doctor(name="D1", specialization="General", department="General", contact="222")
    db.add_all([p, d])
    db.commit()
    return p.id, d.id

def test_create_appointment(client, admin_token, db):
    pid, did = _create_patient_and_doctor(db)
    resp = client.post(
        "/appointments/",
        json={"patient_id": pid, "doctor_id": did, "date_time": (datetime.now() + timedelta(days=1)).isoformat(), "reason": "A"},
        headers=auth_header(admin_token),
    )
    assert resp.status_code == 201
    assert resp.json()["status"] == "scheduled"

def test_list_appointments_filter(client, admin_token, db):
    pid, did = _create_patient_and_doctor(db)
    client.post(
        "/appointments/",
        json={"patient_id": pid, "doctor_id": did, "date_time": datetime.now().isoformat(), "reason": "A"},
        headers=auth_header(admin_token),
    )
    resp = client.get(f"/appointments/?patient_id={pid}", headers=auth_header(admin_token))
    assert resp.status_code == 200
    assert len(resp.json()) == 1

def test_update_appointment_status(client, admin_token, db):
    pid, did = _create_patient_and_doctor(db)
    create = client.post(
        "/appointments/",
        json={"patient_id": pid, "doctor_id": did, "date_time": datetime.now().isoformat(), "reason": "A"},
        headers=auth_header(admin_token),
    )
    aid = create.json()["id"]
    resp = client.patch(f"/appointments/{aid}", json={"status": "completed"}, headers=auth_header(admin_token))
    assert resp.status_code == 200
    assert resp.json()["status"] == "completed"

def test_invalid_appointment_status(client, admin_token, db):
    pid, did = _create_patient_and_doctor(db)
    create = client.post(
        "/appointments/",
        json={"patient_id": pid, "doctor_id": did, "date_time": datetime.now().isoformat(), "reason": "A"},
        headers=auth_header(admin_token),
    )
    aid = create.json()["id"]
    resp = client.patch(f"/appointments/{aid}", json={"status": "invalid"}, headers=auth_header(admin_token))
    assert resp.status_code == 400
```

tests/test_auth.py

```
from tests.conftest import auth_header

def test_health_check(client):
    resp = client.get("/")
    assert resp.status_code == 200
    assert resp.json()["status"] == "healthy"

def test_login_success(client, admin_token):
    assert admin_token is not None

def test_login_wrong_password(client, db):
    from app.models.user import User
    from app.auth import hash_password
    db.add(User(username="test", password_hash=hash_password("pass"), full_name="T", role="admin"))
    db.commit()
    resp = client.post("/auth/login", json={"username": "test", "password": "wrong"})
    assert resp.status_code == 401

def test_register_requires_admin(client, receptionist_token):
    resp = client.post(
        "/auth/register",
        json={"username": "new", "password": "pass", "full_name": "New", "role": "receptionist"},
        headers=auth_header(receptionist_token),
    )
    assert resp.status_code == 403

def test_register_by_admin(client, admin_token):
    resp = client.post(
        "/auth/register",
        json={"username": "new_user", "password": "pass123", "full_name": "New User", "role": "receptionist"},
        headers=auth_header(admin_token),
    )
    assert resp.status_code == 201
    assert resp.json()["username"] == "new_user"

def test_me(client, admin_token):
    resp = client.get("/auth/me", headers=auth_header(admin_token))
    assert resp.status_code == 200
    assert resp.json()["role"] == "admin"
```

tests/test_lambda_reminder.py

```
"""Tests for the appointment-reminder Lambda handler (Phase 2)."""
from __future__ import annotations

import os
from datetime import datetime, timedelta, timezone
from unittest.mock import MagicMock, patch

import pytest

from app.lambda_handlers import appointment_reminder as ar
from app.models.appointment import Appointment
from app.models.doctor import Doctor
from app.models.patient import Patient
from app.models.user import User
from app.auth import hash_password

@pytest.fixture
def seeded_db(db):
    """Seed an admin, a patient, a doctor, and 3 appointments at known offsets."""
    admin = User(username="lambda_admin", password_hash=hash_password("x"), full_name="A", role="admin")
    db.add(admin)
    db.flush()
    patient = Patient(name="P1", age=40, gender="M", contact="9", address="A", blood_group="O+", is_active=True)
    db.add(patient)
    doctor_user = User(username="lambda_doc", password_hash=hash_password("x"), full_name="D", role="doctor")
    db.add(doctor_user)
    db.flush()
    doc = Doctor(user_id=doctor_user.id, name="Dr X", specialization="Gen", department="Gen", contact="9")
    db.add(doc)
    db.flush()

    now = datetime.now(timezone.utc).replace(tzinfo=None)
    a_due = Appointment(patient_id=patient.id, doctor_id=doc.id, date_time=now + timedelta(hours=2), reason="due", status="pending")
    a_far = Appointment(patient_id=patient.id, doctor_id=doc.id, date_time=now + timedelta(hours=72), reason="far", status="pending")
    a_done = Appointment(patient_id=patient.id, doctor_id=doc.id, date_time=now + timedelta(hours=1), reason="done", status="completed")
    db.add_all([a_due, a_far, a_done])
    db.commit()
    return db

def test_find_due_appointments_filters_window_and_status(seeded_db):
    due = ar.find_due_appointments(seeded_db, hours_ahead=24)
    reasons = sorted(a.reason for a in due)
    assert reasons == ["due"], "only the in-window scheduled appointment should match"

def test_lambda_handler_dry_run_when_no_topic(seeded_db, monkeypatch):
    """No SNS_TOPIC_ARN → handler should still run, returning skipped count."""
    monkeypatch.delenv("SNS_TOPIC_ARN", raising=False)

    # The handler does `from app.database import SessionLocal` lazily, so patch at the source module.
    from tests.conftest import TestSession
    with patch("app.database.SessionLocal", new=TestSession):
        result = ar.lambda_handler({}, None)

    assert result["published"] == 0
    assert result["skipped"] == 1
    assert result["ids"] == []

def test_lambda_handler_publishes_to_sns(seeded_db, monkeypatch):
    """When SNS topic is configured, each due appointment is published."""
    monkeypatch.setenv("SNS_TOPIC_ARN", "arn:aws:sns:ap-south-1:111122223333:reminders")

    fake_sns = MagicMock()
    fake_sns.publish.return_value = {"MessageId": "msg-123"}

    from tests.conftest import TestSession

    with patch("app.database.SessionLocal", new=TestSession), \
        patch("boto3.client", return_value=fake_sns):
        result = ar.lambda_handler({}, None)

    assert result["published"] == 1
    assert result["skipped"] == 0
    assert result["ids"] == ["msg-123"]
    fake_sns.publish.assert_called_once()
    # Ensure the published payload is the expected JSON shape
    args, kwargs = fake_sns.publish.call_args
    assert kwargs["TopicArn"].endswith(":reminders")
    assert "appointment_id" in kwargs["Message"]
```

tests/test_logging_middleware.py

```
"""Tests for the structured-logging middleware (Phase 2)."""
from __future__ import annotations

import json
import logging

import pytest

from app.config import settings
from app.logging_middleware import JsonFormatter, configure_logging


def test_json_formatter_emits_valid_json():
    fmt = JsonFormatter()
    record = logging.LogRecord(
        name="app.test",
        level=logging.INFO,
        pathname=__file__,
        lineno=1,
        msg="hello %s",
        args=("world",),
        exc_info=None,
    )
    record.request_id = "abc"
    out = fmt.format(record)
    parsed = json.loads(out)
    assert parsed["msg"] == "hello world"
    assert parsed["level"] == "INFO"
    assert parsed["logger"] == "app.test"
    assert parsed["request_id"] == "abc"
    assert "ts" in parsed


def test_request_id_header_round_trip(client, admin_token):
    """The middleware should accept an inbound X-Request-ID and not crash."""
    headers = {"X-Request-ID": "req-12345", "Authorization": f"Bearer {admin_token}"}
    resp = client.get("/auth/me", headers=headers)
    # Just exercising the middleware path; auth/me returns 200 for admin.
    assert resp.status_code == 200


def test_configure_logging_idempotent(monkeypatch):
    monkeypatch.setattr(settings, "JSON_LOGS", True)
    monkeypatch.setattr(settings, "LOG_LEVEL", "DEBUG")
    configure_logging()
    configure_logging() # second call should not duplicate handlers
    assert len(logging.getLogger().handlers) == 1
```

tests/test_patients.py

```
from tests.conftest import auth_header

PATIENT = {"name": "Test Patient", "age": 30, "gender": "Male", "contact": "9999999999", "address": "Test City", "blood_group": "A"}

def test_create_patient(client, admin_token):
    resp = client.post("/patients/", json=PATIENT, headers=auth_header(admin_token))
    assert resp.status_code == 201
    assert resp.json()["name"] == "Test Patient"

def test_list_patients(client, admin_token):
    client.post("/patients/", json=PATIENT, headers=auth_header(admin_token))
    resp = client.get("/patients/", headers=auth_header(admin_token))
    assert resp.status_code == 200
    assert len(resp.json()) == 1

def test_get_patient(client, admin_token):
    create = client.post("/patients/", json=PATIENT, headers=auth_header(admin_token))
    pid = create.json()["id"]
    resp = client.get(f"/patients/{pid}", headers=auth_header(admin_token))
    assert resp.status_code == 200
    assert resp.json()["id"] == pid

def test_update_patient(client, admin_token):
    create = client.post("/patients/", json=PATIENT, headers=auth_header(admin_token))
    pid = create.json()["id"]
    resp = client.put(f"/patients/{pid}", json={"name": "Updated"}, headers=auth_header(admin_token))
    assert resp.status_code == 200
    assert resp.json()["name"] == "Updated"

def test_delete_patient_soft(client, admin_token):
    create = client.post("/patients/", json=PATIENT, headers=auth_header(admin_token))
    pid = create.json()["id"]
    resp = client.delete(f"/patients/{pid}", headers=auth_header(admin_token))
    assert resp.status_code == 204
    # Should not appear in list anymore
    resp = client.get("/patients/", headers=auth_header(admin_token))
    assert len(resp.json()) == 0

def test_search_patients(client, admin_token):
    client.post("/patients/", json=PATIENT, headers=auth_header(admin_token))
    client.post("/patients/", json={"**PATIENT", "name": "Another Person"}, headers=auth_header(admin_token))
    resp = client.get("/patients/?search=Test", headers=auth_header(admin_token))
    assert len(resp.json()) == 1

def test_doctor_cannot_create_patient(client, doctor_token):
    resp = client.post("/patients/", json=PATIENT, headers=auth_header(doctor_token))
    assert resp.status_code == 403
```

tests/test_storage.py

```
"""Tests for the pluggable storage layer (Phase 2).

Covers:
* LocalStorage round-trip (save → read bytes).
* S3Storage round-trip via the `moto` mocked S3 service.
* Factory selects the correct backend from settings.
"""
from __future__ import annotations

from pathlib import Path

import pytest

from app import storage as storage_mod
from app.config import settings

# -----
# LocalStorage
# -----
def test_local_storage_round_trip(tmp_path: Path):
    backend = storage_mod.LocalStorage(str(tmp_path))
    key = backend.save("report.pdf", b"hello world", content_type="application/pdf")

    p = Path(key)
    assert p.exists(), "file should be written to disk"
    assert p.read_bytes() == b"hello world"
    assert p.suffix == ".pdf"

    # url() for local is just the path
    assert backend.url(key) == key

    backend.delete(key)
    assert not p.exists()

# -----
# S3Storage (mocked with moto)
# -----
moto = pytest.importorskip("moto", reason="install moto to run S3 tests")
boto3 = pytest.importorskip("boto3")

@pytest.fixture
def s3_bucket(monkeypatch):
    """Spin up an in-memory S3, create a bucket, yield its name."""
    from moto import mock_aws

    with mock_aws():
        client = boto3.client("s3", region_name="ap-south-1")
        client.create_bucket(
            Bucket="test-medical-files",
            CreateBucketConfiguration={"LocationConstraint": "ap-south-1"},
        )
        # Make sure factory builds the S3 backend
        monkeypatch.setattr(settings, "STORAGE_BACKEND", "s3")
        monkeypatch.setattr(settings, "S3_BUCKET", "test-medical-files")
        monkeypatch.setattr(settings, "AWS_REGION", "ap-south-1")
        monkeypatch.setattr(settings, "S3_KMS_KEY_ID", "") # use SSE-S3 in tests
        storage_mod.reset_storage()
        yield "test-medical-files"
        storage_mod.reset_storage()

def test_s3_storage_round_trip(s3_bucket):
    backend = storage_mod.get_storage()
    assert isinstance(backend, storage_mod.S3Storage)

    key = backend.save("scan.png", b"PNGDATA", content_type="image/png")
    assert key.startswith("medical-records/")
    assert key.endswith(".png")

    # The object should exist in the mocked S3
    head = backend.client.head_object(Bucket=s3_bucket, Key=key)
    assert head["ContentLength"] == len(b"PNGDATA")

    # Pre-signed URL should be returned and contain the bucket + key
    url = backend.url(key)
    assert s3_bucket in url and key in url

    backend.delete(key)
    with pytest.raises(Exception):
        backend.client.head_object(Bucket=s3_bucket, Key=key)

def test_factory_default_is_local(monkeypatch, tmp_path):
    monkeypatch.setattr(settings, "STORAGE_BACKEND", "local")
    monkeypatch.setattr(settings, "UPLOAD_DIR", str(tmp_path))
    storage_mod.reset_storage()
    backend = storage_mod.get_storage()
    assert isinstance(backend, storage_mod.LocalStorage)

def test_factory_rejects_s3_without_bucket(monkeypatch):
    monkeypatch.setattr(settings, "STORAGE_BACKEND", "s3")
    monkeypatch.setattr(settings, "S3_BUCKET", "")
    storage_mod.reset_storage()
    with pytest.raises(RuntimeError):
        storage_mod.get_storage()
    storage_mod.reset_storage()
```


tests/test_upload_e2e.py

```
"""End-to-end test: upload a medical record file via the new pluggable storage layer."""
from __future__ import annotations

from pathlib import Path

import pytest

from app import storage as storage_mod
from app.auth import hash_password
from app.models.doctor import Doctor
from app.models.medical_record import MedicalRecord
from app.models.patient import Patient
from app.models.user import User

def test_upload_uses_local_backend(client, db, monkeypatch, tmp_path):
    """Smoke test: doctor uploads a PDF; backend writes to tmp dir; record updated."""
    from app.config import settings

    # Force local backend pointing at the test tmp dir
    monkeypatch.setattr(settings, "STORAGE_BACKEND", "local")
    monkeypatch.setattr(settings, "UPLOAD_DIR", str(tmp_path))
    storage_mod.reset_storage()

    # Seed doctor user + doctor profile + patient + medical record
    doc_user = User(username="updoc", password_hash=hash_password("p"), full_name="D", role="doctor")
    db.add(doc_user)
    db.flush()
    doctor = Doctor(user_id=doc_user.id, name="D", specialization="X", department="Y", contact="9")
    db.add(doctor)
    patient = Patient(name="P", age=30, gender="M", contact="9", address="A", blood_group="O+", is_active=True)
    db.add(patient)
    db.flush()
    record = MedicalRecord(patient_id=patient.id, doctor_id=doctor.id, diagnosis="d", prescription="p", notes="n")
    db.add(record)
    db.commit()
    record_id = record.id

    token_resp = client.post("/auth/login", json={"username": "updoc", "password": "p"})
    token = token_resp.json()["access_token"]

    fake_pdf = b"%PDF-1.4 fake content"
    resp = client.post(
        f"/upload/{record_id}",
        headers={"Authorization": f"Bearer {token}"},
        files={"file": ("xray.pdf", fake_pdf, "application/pdf")},
    )
    assert resp.status_code == 200, resp.text
    body = resp.json()
    assert body["backend"] == "local"
    saved = Path(body["file_path"])
    assert saved.exists()
    assert saved.read_bytes() == fake_pdf
    storage_mod.reset_storage()

def test_upload_rejects_disallowed_extension(client, db, monkeypatch, tmp_path):
    from app.config import settings

    monkeypatch.setattr(settings, "STORAGE_BACKEND", "local")
    monkeypatch.setattr(settings, "UPLOAD_DIR", str(tmp_path))
    storage_mod.reset_storage()

    doc_user = User(username="updoc2", password_hash=hash_password("p"), full_name="D", role="doctor")
    db.add(doc_user)
    db.flush()
    doctor = Doctor(user_id=doc_user.id, name="D", specialization="X", department="Y", contact="9")
    db.add(doctor)
    patient = Patient(name="P", age=30, gender="M", contact="9", address="A", blood_group="O+", is_active=True)
    db.add(patient)
    db.flush()
    record = MedicalRecord(patient_id=patient.id, doctor_id=doctor.id, diagnosis="d", prescription="p", notes="n")
    db.add(record)
    db.commit()

    token = client.post("/auth/login", json={"username": "updoc2", "password": "p"}).json()["access_token"]
    resp = client.post(
        f"/upload/{record.id}",
        headers={"Authorization": f"Bearer {token}"},
        files={"file": ("evil.exe", b"MZ", "application/octet-stream")},
    )
    assert resp.status_code == 400
    assert "not allowed" in resp.json()["detail"]
    storage_mod.reset_storage()
```

scripts/generate_code_pdf.py

```
"""Generate a single PDF containing all source code (rubric requirement).
```

```
Usage:
```

```
python scripts/generate_code_pdf.py
```

```
Output:
```

```
report/CC_Project_Code_Group23.pdf
```

```
The PDF preserves filenames as headers and ships every file as  
fixed-width text. We use ReportLab (pure Python, MIT) so contributors  
don't need LaTeX or wkhtmltopdf installed.
```

```
"""
```

```
from __future__ import annotations
```

```
import sys
```

```
from pathlib import Path
```

```
try:
```

```
    from reportlab.lib.pagesizes import A4  
    from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle  
    from reportlab.lib.units import mm  
    from reportlab.platypus import (  
        SimpleDocTemplate, Paragraph, Preformatted, PageBreak, Spacer  
    )
```

```
except ImportError:
```

```
    sys.exit(  
        "reportlab is required:\n python -m pip install reportlab"  
    )
```

```
REPO = Path(__file__).resolve().parent.parent
```

```
INCLUDE_DIRS = [
```

```
    "app",  
    "tests",  
    "scripts",  
    "infra/terraform",  
]
```

```
INCLUDE_FILES = [
```

```
    "Dockerfile",  
    "requirements.txt",  
    "README.md",  
    "report/Phase2_Report.md",  
    "report/Phase2_PPT_Outline.md",  
    "report/Demo_Video_Script.md",  
]
```

```
EXTS = {".py", ".tf", ".sh", ".md", ".txt", ".dockerfile", ""}
```

```
def collect_files() -> list[Path]:
```

```
    files: list[Path] = []  
    for d in INCLUDE_DIRS:  
        for p in sorted((REPO / d).rglob("*")):  
            if p.is_file() and (p.suffix in EXTS or p.name == "Dockerfile"):  
                files.append(p)  
    for f in INCLUDE_FILES:  
        p = REPO / f  
        if p.exists():  
            files.append(p)  
    return files
```

```
def build_pdf(out_path: Path) -> None:
```

```
    out_path.parent.mkdir(parents=True, exist_ok=True)  
    doc = SimpleDocTemplate(  
        str(out_path),  
        pagesize=A4,  
        leftMargin=12 * mm, rightMargin=12 * mm,  
        topMargin=12 * mm, bottomMargin=12 * mm,  
        title="Cloud-Native Healthcare DMS – Source Code (Group 23)",  
    )  
    styles = getSampleStyleSheet()  
    code_style = ParagraphStyle(  
        "Code", parent=styles["Code"],  
        fontName="Courier", fontSize=7, leading=8,  
    )  
    h1 = styles["Heading1"]  
    h2 = styles["Heading2"]  
    body = styles["BodyText"]  
  
    story = []  
    story.append(Paragraph("Cloud-Native Healthcare Data Management System", h1))  
    story.append(Paragraph("Phase 2 – Complete Source Code", h2))  
    story.append(Paragraph(  
        "BITS Pilani · Cloud Computing (CSIZG527) · Group 23<br/>"  
        "Karan Rawat · Vikas Kumar · Kriti Tripathi · Avnish Pratap Singh",  
        body,  
    ))  
    story.append(PageBreak())  
  
    for f in collect_files():  
        rel = f.relative_to(REPO).as_posix()  
        story.append(Paragraph(f"<b>{rel}</b>", h2))  
        try:  
            text = f.read_text(encoding="utf-8")  
        except UnicodeDecodeError:  
            text = f.read_bytes().decode("utf-8", errors="replace")  
        # Keep page-break friendly: split on blank lines so reportlab can flow.  
        for chunk in text.split("\n\n"):
```

```
        if not chunk.strip():
            continue
        # Escape < > & for reportlab Preformatted is not needed; it's literal.
        story.append(Preformatted(chunk, code_style))
        story.append(Spacer(1, 2 * mm))
        story.append(PageBreak())

    doc.build(story)
    print(f"✓ Wrote {out_path} ({out_path.stat().st_size / 1024:.0f} KB)")

if __name__ == "__main__":
    build_pdf(REPO / "report" / "CC_Project_Code_Group23.pdf")
```

scripts/seed.py

```
"""Seed script – populates DB with realistic demo data (100 patients, 10 doctors)."""

import sys
import random
from pathlib import Path
from datetime import datetime, timedelta

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from app.database import engine, Base, SessionLocal
from app.models.user import User
from app.models.patient import Patient
from app.models.doctor import Doctor
from app.models.appointment import Appointment
from app.models.medical_record import MedicalRecord
from app.auth import hash_password

# ■■■ Realistic Indian name pools ■■■
FIRST_MALE = [
    "Aarav", "Arjun", "Rahul", "Vikram", "Rohit", "Aditya", "Siddharth", "Karan",
    "Amit", "Rajesh", "Suresh", "Manish", "Deepak", "Nikhil", "Abhishek", "Gaurav",
    "Varun", "Harsh", "Pranav", "Ishaan", "Yash", "Akash", "Vivek", "Ravi",
    "Neeraj", "Sahil", "Tarun", "Mohit", "Ankit", "Ashwin", "Dhruv", "Kunal",
    "Pankaj", "Sandeep", "Vishal", "Jatin", "Mukesh", "Naveen", "Ramesh", "Sunil",
]
FIRST_FEMALE = [
    "Priya", "Ananya", "Sneha", "Kavya", "Neha", "Pooja", "Riya", "Divya",
    "Sakshi", "Anjali", "Shreya", "Meera", "Kriti", "Nisha", "Aishwarya", "Swati",
    "Pallavi", "Tanvi", "Simran", "Bhavna", "Isha", "Ritika", "Sanya", "Zara",
    "Deepika", "Jyoti", "Komal", "Lavanya", "Mira", "Nikita", "Payal", "Rachna",
    "Shalini", "Tanya", "Uma", "Varsha", "Yamini", "Aditi", "Bhavika", "Chitra",
]
LAST_NAMES = [
    "Sharma", "Verma", "Gupta", "Singh", "Kumar", "Patel", "Reddy", "Iyer",
    "Nair", "Joshi", "Mehta", "Shah", "Rao", "Das", "Mukherjee", "Chatterjee",
    "Bose", "Pillai", "Menon", "Agarwal", "Chauhan", "Yadav", "Thakur", "Saxena",
    "Malhotra", "Kapoor", "Bansal", "Bhatt", "Mishra", "Pandey", "Tiwari", "Dubey",
]
CITIES = [
    "Delhi", "Mumbai", "Bangalore", "Hyderabad", "Chennai", "Kolkata", "Pune",
    "Jaipur", "Lucknow", "Ahmedabad", "Chandigarh", "Bhopal", "Indore", "Nagpur",
    "Patna", "Thiruvananthapuram", "Kochi", "Coimbatore", "Visakhapatnam", "Surat",
]
BLOOD_GROUPS = ["A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-"]
GENDERS = ["Male", "Female"]

# ■■■ Doctor data ■■■
DOCTORS = [
    ("Dr. Priya Sharma", "General Medicine", "General Medicine", "dr_sharma"),
    ("Dr. Amit Patel", "Cardiology", "Cardiology", None),
    ("Dr. Sneha Reddy", "Dermatology", "Dermatology", None),
    ("Dr. Rajesh Kumar", "Orthopedics", "Orthopedics", None),
    ("Dr. Kavya Nair", "Pediatrics", "Pediatrics", None),
    ("Dr. Siddharth Joshi", "ENT", "ENT", None),
    ("Dr. Meera Iyer", "Gynecology", "Gynecology", None),
    ("Dr. Arjun Mehta", "Neurology", "Neurology", None),
    ("Dr. Ananya Gupta", "Ophthalmology", "Ophthalmology", None),
    ("Dr. Vikram Rao", "General Surgery", "Surgery", None),
]
REASONS = [
    "Routine checkup", "Fever and cold", "Persistent headache", "Back pain",
    "Skin rash", "Chest discomfort", "Eye irritation", "Follow-up visit",
    "Joint pain", "Difficulty breathing", "Stomach ache", "Ear infection",
    "High blood pressure review", "Diabetes management", "Allergy symptoms",
    "Vaccination", "Pregnancy checkup", "Lab results review", "Cough and sore throat",
    "Knee pain", "Dizziness", "Fatigue", "Weight loss consultation",
    "Dental referral follow-up", "Post-surgery review", "Annual physical",
]
DIAGNOSES = [
    "Upper respiratory tract infection", "Mild hypertension", "Type 2 Diabetes (controlled)",
    "Vitamin D deficiency", "Iron deficiency anemia", "Acute gastritis",
    "Lumbar strain", "Contact dermatitis", "Allergic rhinitis", "Migraine",
    "Conjunctivitis", "Otitis media", "Osteoarthritis", "GERD",
    "Urinary tract infection", "Bronchitis", "Cervical spondylosis",
    "Hypothyroidism", "Anxiety disorder", "Mild angina",
]
PRESCRIPTIONS = [
    "Paracetamol 500mg TDS x 5 days", "Amoxicillin 500mg TDS x 7 days",
    "Omeprazole 20mg OD x 14 days", "Metformin 500mg BD",
    "Amlodipine 5mg OD", "Vitamin D3 60000 IU weekly x 8 weeks",
    "Iron tablets OD x 30 days", "Cetirizine 10mg OD x 10 days",
    "Ibuprofen 400mg BD x 5 days", "Pantoprazole 40mg OD x 14 days",
    "Levothyroxine 50mcg OD", "Aspirin 75mg OD", "Atorvastatin 10mg OD",
    "Azithromycin 500mg OD x 3 days", "Cough syrup 10ml TDS x 5 days",
]

def seed():
    Base.metadata.create_all(bind=engine)
    db = SessionLocal()

    if db.query(User).filter(User.username == "admin").first():
        print("Database already seeded. Skipping.")
```

```

    db.close()
    return

random.seed(42) # Reproducible data

# ■■■ Users ■■■
admin = User(username="admin", password_hash=hash_password("admin123"),
             full_name="Admin User", role="admin")
doc_user = User(username="dr_sharma", password_hash=hash_password("doctor123"),
               full_name="Dr. Priya Sharma", role="doctor")
recep_user = User(username="reception1", password_hash=hash_password("recep123"),
                  full_name="Ravi Kumar", role="receptionist")
db.add_all([admin, doc_user, recep_user])
db.flush()

# ■■■ 10 Doctors ■■■
doctors = []
for name, spec, dept, uname in DOCTORS:
    uid = doc_user.id if uname == "dr_sharma" else None
    d = Doctor(
        user_id=uid, name=name, specialization=spec,
        department=dept, contact=f"98{random.randint(10000000, 99999999)}"
    )
    db.add(d)
    doctors.append(d)
db.flush()
print(f" Created {len(doctors)} doctors")

# ■■■ 100 Patients ■■■
patients = []
for i in range(100):
    gender = random.choice(GENDERS)
    first = random.choice(FIRST_MALE if gender == "Male" else FIRST_FEMALE)
    last = random.choice(LAST_NAMES)
    p = Patient(
        name=f"{first} {last}",
        age=random.randint(5, 85),
        gender=gender,
        contact=f"9{random.randint(100000000, 999999999)}",
        address=random.choice(CITIES),
        blood_group=random.choice(BLOOD_GROUPS),
    )
    db.add(p)
    patients.append(p)
db.flush()
print(f" Created {len(patients)} patients")

# ■■■ ~200 Appointments (2 per patient avg, spread over past 30 days + next 15 days) ■■■
now = datetime.now()
appointments = []
statuses_past = ["completed", "completed", "completed", "cancelled"] # 75% completed for past
statuses_future = ["scheduled"]

for p in patients:
    num_appts = random.randint(1, 4)
    for _ in range(num_appts):
        days_offset = random.randint(-30, 15)
        dt = now + timedelta(days=days_offset, hours=random.randint(8, 17), minutes=random.choice([0, 15, 30, 45]))
        is_past = days_offset < 0
        status = random.choice(statuses_past) if is_past else random.choice(statuses_future)
        a = Appointment(
            patient_id=p.id,
            doctor_id=random.choice(doctors).id,
            date_time=dt,
            reason=random.choice(REASONS),
            status=status,
        )
        db.add(a)
        appointments.append(a)
db.flush()
print(f" Created {len(appointments)} appointments")

# ■■■ Medical records for completed appointments ■■■
records_count = 0
for a in appointments:
    if a.status == "completed" and random.random() < 0.8: # 80% of completed get a record
        r = MedicalRecord(
            patient_id=a.patient_id,
            doctor_id=a.doctor_id,
            diagnosis=random.choice(DIAGNOSES),
            prescription=random.choice(PRESCRIPTIONS),
            notes=random.choice([
                "Patient responding well to treatment",
                "Follow-up in 2 weeks recommended",
                "Lab tests ordered – CBC, LFT, KFT",
                "Vitals normal. Continue current medication",
                "Referred to specialist for further evaluation",
                "Patient advised lifestyle modifications",
                "Symptoms improved since last visit",
                "X-ray results normal. Rest advised",
                "Blood sugar levels within range",
                "Blood pressure stable on current dosage",
            ]),
        )
        db.add(r)
        records_count += 1

```

```
db.commit()
db.close()

print(f" Created {records_count} medical records")
print()
print("Database seeded successfully!")
print(" Admin login:      admin / admin123")
print(" Doctor login:     dr_sharma / doctor123")
print(" Receptionist:      reception1 / recep123")
print(f" Total: {len(patients)} patients, {len(doctors)} doctors, {len(appointments)} appointments, {records_count} records")

if __name__ == "__main__":
    seed()
```

infra/terraform/compute_ec2.tf

```
#####
# Compute layer: launch template + ASG + ALB + target group.
#
# - EC2 instances boot from user_data.sh, clone the repo, and start uvicorn.
# - ALB health check hits GET / (the FastAPI health_check endpoint).
# - ASG scales on CPU > 60% (TargetTracking).
#####

resource "aws_launch_template" "app" {
  name_prefix      = "${local.name}-lt-"
  image_id         = data.aws_ami.al2023.id
  instance_type    = var.instance_type
  key_name         = var.key_pair_name == "" ? null : var.key_pair_name
  vpc_security_group_ids = [aws_security_group.app.id]

  iam_instance_profile { name = aws_iam_instance_profile.ec2.name }

  metadata_options {
    http_tokens      = "required" # IMDSv2 only
    http_endpoint    = "enabled"
    http_put_response_hop_limit = 2
  }

  user_data = base64encode(templatefile("${path.module}/user_data.sh", {
    repo_url      = var.app_repo_url
    repo_ref      = var.app_repo_ref
    db_host       = aws_db_instance.main.address
    db_port       = aws_db_instance.main.port
    db_name       = var.db_name
    db_username   = var.db_username
    db_password   = var.db_password
    s3_bucket     = aws_s3_bucket.files.bucket
    aws_region    = var.aws_region
    sns_topic_arn = aws_sns_topic.reminders.arn
    kms_key_arn   = aws_kms_key.data.arn
  }))

  tag_specifications {
    resource_type = "instance"
    tags          = { Name = "${local.name}-app" }
  }
}

resource "aws_lb" "app" {
  name                = "${local.name}-alb"
  load_balancer_type = "application"
  subnets            = aws_subnet.public[*].id
  security_groups     = [aws_security_group.alb.id]
}

resource "aws_lb_target_group" "app" {
  name        = "${local.name}-tg"
  port        = local.app_port
  protocol    = "HTTP"
  vpc_id      = aws_vpc.main.id
  target_type = "instance"

  health_check {
    path          = "/"
    interval      = 30
    timeout       = 5
    healthy_threshold = 2
    unhealthy_threshold = 3
    matcher       = "200"
  }

  deregistration_delay = 30
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}

resource "aws_autoscaling_group" "app" {
  name                = "${local.name}-asg"
  min_size            = var.asg_min
  max_size            = var.asg_max
  desired_capacity    = var.asg_desired
  vpc_zone_identifier = aws_subnet.private[*].id
  target_group_arns   = [aws_lb_target_group.app.arn]
  health_check_type   = "ELB"
  health_check_grace_period = 180

  launch_template {
    id      = aws_launch_template.app.id
    version = "$Latest"
  }

  tag {
```

```
    key          = "Name"
    value         = "${local.name}-app"
    propagate_at_launch = true
  }
}

resource "aws_autoscaling_policy" "cpu_target" {
  name                = "${local.name}-cpu-target"
  autoscaling_group_name = aws_autoscaling_group.app.name
  policy_type         = "TargetTrackingScaling"

  target_tracking_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ASGAverageCPUUtilization"
    }
  }
  target_value = 60.0
}
}
```


infra/terraform/database_rds.tf

```
#####
# RDS PostgreSQL – Multi-AZ, encrypted, in private subnets.
#####

resource "aws_db_subnet_group" "main" {
  name       = "${local.name}-db-subnets"
  subnet_ids = aws_subnet.private[*].id
  tags       = { Name = "${local.name}-db-subnets" }
}

resource "aws_db_instance" "main" {
  identifier      = "${local.name}-postgres"
  engine          = "postgres"
  engine_version  = "15.7"
  instance_class  = var.db_instance_class
  allocated_storage = 20
  storage_type     = "gp3"
  storage_encrypted = true
  kms_key_id       = aws_kms_key.data.arn

  db_name      = var.db_name
  username     = var.db_username
  password     = var.db_password

  multi_az      = true
  publicly_accessible = false
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.db.id]

  backup_retention_period = 7
  skip_final_snapshot     = true    # academic project
  deletion_protection     = false
  apply_immediately       = true

  performance_insights_enabled = true
  performance_insights_kms_key_id = aws_kms_key.data.arn

  tags = { Name = "${local.name}-postgres" }
}
```

infra/terraform/lambda_reminder.tf

```
#####
# Lambda (appointment_reminder) + EventBridge schedule + SNS topic.
#####

resource "aws_sns_topic" "reminders" {
  name           = "${local.name}-reminders"
  kms_master_key_id = aws_kms_key.data.arn
}

# Optional email subscription so demo viewers see real notifications.
resource "aws_sns_topic_subscription" "alarm_email" {
  count = var.alarm_email == "" ? 0 : 1
  topic_arn = aws_sns_topic.reminders.arn
  protocol = "email"
  endpoint = var.alarm_email
}

# Package the app into a zip Lambda can run. We include the whole `app/`
# package so models + db code are importable inside the function.
data "archive_file" "lambda" {
  type = "zip"
  source_dir = "${path.module}/../app"
  output_path = "${path.module}/build/appointment_reminder.zip"
}

resource "aws_lambda_function" "reminder" {
  function_name = "${local.name}-reminder"
  role          = aws_iam_role.lambda.arn
  runtime       = "python3.11"
  handler       = "lambda_handlers.appointment_reminder.lambda_handler"
  filename      = data.archive_file.lambda.output_path
  source_code_hash = data.archive_file.lambda.output_base64sha256
  timeout       = 30
  memory_size   = 256

  vpc_config {
    subnet_ids = aws_subnet.private[*].id
    security_group_ids = [aws_security_group.lambda.id]
  }

  environment {
    variables = {
      DATABASE_URL      = "postgresql+psycopg2://${var.db_username}:${var.db_password}@${aws_db_instance.main.address}:"
      SNS_TOPIC_ARN     = aws_sns_topic.reminders.arn
      REMINDER_WINDOW_HOURS = "24"
      JSON_LOGS         = "true"
      LOG_LEVEL         = "INFO"
    }
  }
}

# Trigger Lambda every 15 minutes via EventBridge.
resource "aws_cloudwatch_event_rule" "every_15_min" {
  name = "${local.name}-reminder-schedule"
  schedule_expression = "rate(15 minutes)"
}

resource "aws_cloudwatch_event_target" "reminder" {
  rule = aws_cloudwatch_event_rule.every_15_min.name
  target_id = "reminder-lambda"
  arn = aws_lambda_function.reminder.arn
}

resource "aws_lambda_permission" "events" {
  statement_id = "AllowEventBridgeInvoke"
  action       = "lambda:InvokeFunction"
  function_name = aws_lambda_function.reminder.function_name
  principal     = "events.amazonaws.com"
  source_arn    = aws_cloudwatch_event_rule.every_15_min.arn
}
```

infra/terraform/main.tf

```
terraform {
  required_version = ">= 1.5"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.50"
    }
    archive = {
      source = "hashicorp/archive"
      version = "~> 2.4"
    }
  }
}

provider "aws" {
  region = var.aws_region
  default_tags {
    tags = {
      Project      = var.project
      Environment  = var.environment
      Owner        = "BITS-Group23"
      Course       = "CSIZG527"
      ManagedBy    = "Terraform"
    }
  }
}

data "aws_availability_zones" "available" {
  state = "available"
}

# Latest Amazon Linux 2023 AMI for x86_64
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

locals {
  azs      = slice(data.aws_availability_zones.available.names, 0, 2)
  name     = "${var.project}-${var.environment}"
  app_port = 8000
  s3_bucket = "${var.project}-${var.environment}-files-${random_id.suffix.hex}"
}

resource "random_id" "suffix" {
  byte_length = 3
}
```

infra/terraform/network.tf

```
#####
# VPC, subnets, internet & NAT gateways, route tables.
#
# Public subnets host the ALB and the NAT GW.
# Private subnets host the EC2 ASG and the RDS instance.
#####

resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true
  tags            = { Name = "${local.name}-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "${local.name}-igw" }
}

resource "aws_subnet" "public" {
  count            = length(var.public_subnet_cidrs)
  vpc_id           = aws_vpc.main.id
  cidr_block       = var.public_subnet_cidrs[count.index]
  availability_zone = local.azs[count.index]
  map_public_ip_on_launch = true
  tags = {
    Name = "${local.name}-public-${count.index}"
    Tier = "public"
  }
}

resource "aws_subnet" "private" {
  count            = length(var.private_subnet_cidrs)
  vpc_id           = aws_vpc.main.id
  cidr_block       = var.private_subnet_cidrs[count.index]
  availability_zone = local.azs[count.index]
  tags = {
    Name = "${local.name}-private-${count.index}"
    Tier = "private"
  }
}

# Single NAT GW (cost-conscious for an academic project).
resource "aws_eip" "nat" {
  domain = "vpc"
  tags   = { Name = "${local.name}-nat-eip" }
}

resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public[0].id
  tags          = { Name = "${local.name}-nat" }
  depends_on    = [aws_internet_gateway.igw]
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "${local.name}-public-rt" }
}

resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat.id
  }
  tags = { Name = "${local.name}-private-rt" }
}

resource "aws_route_table_association" "public" {
  count            = length(aws_subnet.public)
  subnet_id       = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "private" {
  count            = length(aws_subnet.private)
  subnet_id       = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private.id
}
```

infra/terraform/observability.tf

```
#####
# CloudTrail (audit) + CloudWatch alarms.
#####

# ---- Dedicated bucket for CloudTrail logs -----
resource "aws_s3_bucket" "trail" {
  bucket = "${local.name}-cloudtrail-${random_id.suffix.hex}"
  force_destroy = true
}

resource "aws_s3_bucket_public_access_block" "trail" {
  bucket = aws_s3_bucket.trail.id
  block_public_acls = true
  block_public_policy = true
  ignore_public_acls = true
  restrict_public_buckets = true
}

data "aws_caller_identity" "me" {}

data "aws_iam_policy_document" "trail_bucket" {
  statement {
    sid = "AWSCloudTrailAcLCheck"
    actions = ["s3:GetBucketAcl"]
    principals {
      type = "Service"
      identifiers = ["cloudtrail.amazonaws.com"]
    }
    resources = [aws_s3_bucket.trail.arn]
  }
  statement {
    sid = "AWSCloudTrailWrite"
    actions = ["s3:PutObject"]
    principals {
      type = "Service"
      identifiers = ["cloudtrail.amazonaws.com"]
    }
    resources = ["${aws_s3_bucket.trail.arn}/AWSLogs/${data.aws_caller_identity.me.account_id}/*"]
    condition {
      test = "StringEquals"
      variable = "s3:x-amz-acl"
      values = ["bucket-owner-full-control"]
    }
  }
}

resource "aws_s3_bucket_policy" "trail" {
  bucket = aws_s3_bucket.trail.id
  policy = data.aws_iam_policy_document.trail_bucket.json
}

resource "aws_cloudtrail" "main" {
  name = "${local.name}-trail"
  s3_bucket_name = aws_s3_bucket.trail.id
  include_global_service_events = true
  is_multi_region_trail = true
  enable_log_file_validation = true
  kms_key_id = aws_kms_key.data.arn
  depends_on = [aws_s3_bucket_policy.trail]
}

# ---- CloudWatch log group for the app -----
resource "aws_cloudwatch_log_group" "app" {
  name = "/aws/ec2/healthdms"
  retention_in_days = 30
  kms_key_id = aws_kms_key.data.arn
}

# ---- Alarms -----
resource "aws_cloudwatch_metric_alarm" "alb_5xx" {
  alarm_name = "${local.name}-alb-5xx"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods = 2
  metric_name = "HTTPCode_Target_5XX_Count"
  namespace = "AWS/ApplicationELB"
  period = 60
  statistic = "Sum"
  threshold = 5
  alarm_description = "More than 5 5xx responses per minute from the app target group"
  dimensions = { LoadBalancer = aws_lb.app.arn_suffix }
  alarm_actions = [aws_sns_topic.reminders.arn]
  treat_missing_data = "notBreaching"
}

resource "aws_cloudwatch_metric_alarm" "asg_high_cpu" {
  alarm_name = "${local.name}-asg-cpu"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods = 3
  metric_name = "CPUUtilization"
  namespace = "AWS/EC2"
  period = 60
  statistic = "Average"
  threshold = 80
  alarm_description = "ASG average CPU > 80% for 3 consecutive minutes"
}
```

```
    dimensions      = { AutoScalingGroupName = aws_autoscaling_group.app.name }
    alarm_actions    = [aws_sns_topic.reminders.arn]
  }
```

infra/terraform/outputs.tf

```
output "alb_url" {
  description = "Open this URL in a browser for the live dashboard."
  value       = "http://${aws_lb.app.dns_name}/dashboard"
}

output "alb_dns" {
  value = aws_lb.app.dns_name
}

output "rds_endpoint" {
  description = "RDS PostgreSQL endpoint (host:port).\"
  value       = "${aws_db_instance.main.address}:${aws_db_instance.main.port}"
  sensitive   = true
}

output "s3_bucket" {
  value = aws_s3_bucket.files.bucket
}

output "kms_key_arn" {
  value = aws_kms_key.data.arn
}

output "sns_topic_arn" {
  value = aws_sns_topic.reminders.arn
}

output "lambda_function" {
  value = aws_lambda_function.reminder.function_name
}

output "cloudtrail_bucket" {
  value = aws_s3_bucket.trail.bucket
}

output "log_group" {
  value = aws_cloudwatch_log_group.app.name
}
```

infra/terraform/README.md

Cloud-Native Healthcare DMS – Terraform Infrastructure (Phase 2)

This Terraform configuration provisions the entire AWS Phase-2 stack for Group 23's Cloud Computing project. One `terraform apply` brings up:

Layer	AWS Resources
Networking	VPC (`10.0.0.0/16`), 2× public subnets, 2× private subnets, IGW, NAT, route tables
Security	KMS CMK (RDS + S3), IAM roles, security groups, S3 SSE-KMS, RDS encryption
Storage	S3 bucket (versioned, KMS-encrypted, public access blocked, lifecycle rules)
Database	RDS PostgreSQL 15, Multi-AZ, encrypted, in private subnets
Compute	EC2 Launch Template + Auto Scaling Group (min 1, max 3) behind ALB
Load Balancer	Internet-facing Application Load Balancer with HTTP listener + target group
Auto-scaling	TargetTracking on average CPU = 60%
Serverless	Lambda (appointment reminder) on a 15-minute EventBridge schedule
Notifications	SNS topic for appointment reminders
Audit	CloudTrail (all-region, KMS-encrypted, S3 backed)
Monitoring	CloudWatch log group + 2 alarms (5xx, high CPU) wired to SNS

Layout

...

```
infra/terraform/
■■■ README.md      ← this file
■■■ main.tf         ← provider, data sources, locals
■■■ variables.tf    ← inputs (region, project name, db creds, etc.)
■■■ outputs.tf      ← ALB DNS, RDS endpoint, S3 bucket
■■■ network.tf      ← VPC, subnets, IGW, NAT, route tables
■■■ security.tf     ← KMS, IAM, security groups
■■■ storage_s3.tf   ← S3 bucket for medical files
■■■ database_rds.tf ← RDS PostgreSQL Multi-AZ
■■■ compute_ec2.tf  ← Launch template, ASG, ALB, target group
■■■ lambda_reminder.tf ← Lambda + EventBridge + SNS topic
■■■ observability.tf ← CloudTrail, CloudWatch alarms
■■■ user_data.sh    ← EC2 bootstrap (clones repo, runs uvicorn)
...
```

Prerequisites

```
``bash
# 1. AWS account with admin privileges
# 2. AWS CLI configured (`aws configure`) or env vars set
# 3. Terraform ≥ 1.5
# 4. An existing EC2 key pair name (for SSH if you need to debug nodes)
...
```

Usage

```
``bash
cd infra/terraform

# (One-time) initialise the providers
terraform init

# Inspect the plan
terraform plan -var "db_password=ChangeMe123!" -var "key_pair_name=my-key"

# Apply (~12-15 minutes – RDS is the slowest)
terraform apply -auto-approve \
  -var "db_password=ChangeMe123!" \
  -var "key_pair_name=my-key"

# Outputs include the ALB URL – open it in a browser
terraform output alb_url

# When done – TEAR IT DOWN to avoid AWS charges
terraform destroy -auto-approve -var "db_password=ChangeMe123!" -var "key_pair_name=my-key"
...
```

What the EC2 user_data does

`user\_data.sh` runs on first boot for every ASG instance:

1. Installs Python 3.11, git, pip, the CloudWatch agent.
2. Clones `https://github.com/avanishpsingh/cloud-health-dms` into /opt/app`.`
3. Writes `/opt/app/.env` from EC2 instance metadata + Terraform-injected values (RDS endpoint, S3 bucket, KMS key, SNS topic, JSON_LOGS=True).`
4. Installs `requirements.txt` (+ `boto3`).`
5. Starts `uvicorn` as a systemd` unit (healthdms.service`) listening on :8000`.`
6. Starts the CloudWatch agent so logs flow to `/aws/ec2/healthdms`.`

Cost estimate (us-east-1, on-demand)

Resource	~Monthly cost (USD)
EC2 t3.micro × 2	\$15
ALB	\$18
RDS db.t3.micro Multi-AZ	\$30
S3 (1 GB + few thousand requests)	< \$1
Lambda + CloudWatch + CloudTrail	< \$1
NAT Gateway	\$32
Total	**~\$95 / month**

> ■■ Run `terraform destroy` immediately after the demo / viva.

Production hardening (out of scope for this assignment)

- Add HTTPS listener with ACM certificate + Route 53 alias.

- Move secrets to AWS Secrets Manager.
- Add WAF on the ALB.
- Enable AWS Config conformance pack for HIPAA.
- Convert state to remote backend (S3 + DynamoDB lock).

infra/terraform/security.tf

```
#####
# Security: KMS CMK, IAM roles for EC2 + Lambda, and security groups.
#
# - KMS key encrypts both RDS and the S3 medical-files bucket (Objective 03).
# - EC2 instance role grants S3 read/write + KMS decrypt + CloudWatch Logs put.
# - Lambda role grants RDS network access (via SG) + SNS publish + logs.
#####

# ---- KMS CMK shared by RDS + S3 -----
resource "aws_kms_key" "data" {
  description      = "${local.name} - encrypts S3 medical files & RDS storage"
  enable_key_rotation = true
  deletion_window_in_days = 7
  tags             = { Name = "${local.name}-kms" }
}

resource "aws_kms_alias" "data" {
  name          = "alias/${local.name}-data"
  target_key_id = aws_kms_key.data.key_id
}

# ---- Security groups -----
resource "aws_security_group" "alb" {
  name        = "${local.name}-alb-sg"
  description = "Public ingress to the ALB on 80/443"
  vpc_id      = aws_vpc.main.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "app" {
  name        = "${local.name}-app-sg"
  description = "FastAPI app - only ALB may hit :8000"
  vpc_id      = aws_vpc.main.id

  ingress {
    from_port = local.app_port
    to_port   = local.app_port
    protocol  = "tcp"
    security_groups = [aws_security_group.alb.id]
  }

  # SSH from anywhere - disable in prod, here for viva debug only.
  dynamic "ingress" {
    for_each = var.key_pair_name == "" ? [] : [1]
    content {
      from_port = 22
      to_port   = 22
      protocol  = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "db" {
  name        = "${local.name}-db-sg"
  description = "RDS PostgreSQL - only the app SG may connect"
  vpc_id      = aws_vpc.main.id

  ingress {
    from_port = 5432
    to_port   = 5432
    protocol  = "tcp"
    security_groups = [aws_security_group.app.id, aws_security_group.lambda.id]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
  }
}
```

```

        cidr_blocks = ["0.0.0.0/0"]
    }
}

resource "aws_security_group" "lambda" {
    name = "${local.name}-lambda-sg"
    description = "Lambda ENIs in the VPC – egress only"
    vpc_id = aws_vpc.main.id

    egress {
        from_port = 0
        to_port = 0
        protocol = "-1"
        cidr_blocks = ["0.0.0.0/0"]
    }
}

# ---- IAM: EC2 instance role -----
data "aws_iam_policy_document" "ec2_assume" {
    statement {
        actions = ["sts:AssumeRole"]
        principals {
            type = "Service"
            identifiers = ["ec2.amazonaws.com"]
        }
    }
}

resource "aws_iam_role" "ec2" {
    name = "${local.name}-ec2-role"
    assume_role_policy = data.aws_iam_policy_document.ec2_assume.json
}

resource "aws_iam_role_policy_attachment" "ec2_ssm" {
    role = aws_iam_role.ec2.name
    policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}

resource "aws_iam_role_policy_attachment" "ec2_cwagent" {
    role = aws_iam_role.ec2.name
    policy_arn = "arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy"
}

data "aws_iam_policy_document" "ec2_inline" {
    statement {
        sid = "S3MedicalBucket"
        actions = ["s3:GetObject", "s3:PutObject", "s3:DeleteObject", "s3:ListBucket"]
        resources = [
            aws_s3_bucket.files.arn,
            "${aws_s3_bucket.files.arn}/*",
        ]
    }
    statement {
        sid = "KMSEncryptDecrypt"
        actions = ["kms:Encrypt", "kms:Decrypt", "kms:GenerateDataKey", "kms:DescribeKey"]
        resources = [aws_kms_key.data.arn]
    }
    statement {
        sid = "PublishReminders"
        actions = ["sns:Publish"]
        resources = [aws_sns_topic.reminders.arn]
    }
}

resource "aws_iam_role_policy" "ec2_inline" {
    name = "${local.name}-ec2-inline"
    role = aws_iam_role.ec2.id
    policy = data.aws_iam_policy_document.ec2_inline.json
}

resource "aws_iam_instance_profile" "ec2" {
    name = "${local.name}-ec2-profile"
    role = aws_iam_role.ec2.name
}

# ---- IAM: Lambda execution role -----
data "aws_iam_policy_document" "lambda_assume" {
    statement {
        actions = ["sts:AssumeRole"]
        principals {
            type = "Service"
            identifiers = ["lambda.amazonaws.com"]
        }
    }
}

resource "aws_iam_role" "lambda" {
    name = "${local.name}-lambda-role"
    assume_role_policy = data.aws_iam_policy_document.lambda_assume.json
}

resource "aws_iam_role_policy_attachment" "lambda_basic" {
    role = aws_iam_role.lambda.name
    policy_arn = "arn:aws:iam::aws:policy/service-role/AWSLambdaVPCAccessExecutionRole"
}

data "aws_iam_policy_document" "lambda_inline" {
    statement {
        sid = "PublishReminders"
    }
}

```

```
        actions    = ["sns:Publish"]
        resources = [aws_sns_topic.reminders.arn]
    }
}

resource "aws_iam_role_policy" "lambda_inline" {
    name     = "${local.name}-lambda-inline"
    role     = aws_iam_role.lambda.id
    policy   = data.aws_iam_policy_document.lambda_inline.json
}
```

infra/terraform/storage_s3.tf

```
#####
# S3 bucket for medical files (Phase 2 replacement for ./uploads).
#
# - SSE-KMS using the shared CMK (Objective 03).
# - Versioning ON for tamper-evidence.
# - Public access blocked at the account level.
# - Lifecycle rule: transition to IA after 30d, Glacier after 90d, expire after 7y
#   (HIPAA minimum retention).
#####

resource "aws_s3_bucket" "files" {
  bucket          = local.s3_bucket
  force_destroy   = true # academic project - destroy on `terraform destroy`
  tags            = { Name = "${local.name}-medical-files" }
}

resource "aws_s3_bucket_public_access_block" "files" {
  bucket          = aws_s3_bucket.files.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_versioning" "files" {
  bucket = aws_s3_bucket.files.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "files" {
  bucket = aws_s3_bucket.files.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
      kms_master_key_id = aws_kms_key.data.arn
    }
    bucket_key_enabled = true
  }
}

resource "aws_s3_bucket_lifecycle_configuration" "files" {
  bucket = aws_s3_bucket.files.id
  rule {
    id = "phi-retention"
    status = "Enabled"
    filter {}
    transition {
      days          = 30
      storage_class = "STANDARD_IA"
    }
    transition {
      days          = 90
      storage_class = "GLACIER"
    }
    expiration {
      days = 2555 # 7 years (HIPAA / DISHA minimum retention)
    }
  }
}
```

infra/terraform/user_data.sh

```
#!/usr/bin/env bash
# EC2 user-data – bootstraps the FastAPI app on Amazon Linux 2023.
# Variables in ${...} are interpolated by Terraform's templatefile().
set -euxo pipefail

# 1. Base packages
dnf install -y python3.11 python3.11-pip git amazon-cloudwatch-agent

# 2. Clone the application repo
mkdir -p /opt/app
git clone --depth 1 --branch ${repo_ref} ${repo_url} /opt/app

# 3. Python venv + deps (boto3 added for S3/SNS, psycpg2-binary for RDS)
python3.11 -m venv /opt/app/venv
/opt/app/venv/bin/pip install --upgrade pip
/opt/app/venv/bin/pip install -r /opt/app/requirements.txt
/opt/app/venv/bin/pip install boto3 psycpg2-binary

# 4. Environment file consumed by app/config.py via pydantic-settings
cat > /opt/app/.env <<EOF
DATABASE_URL=postgresql+psycpg2://${db_username}:${db_password}@${db_host}:${db_port}/${db_name}
SECRET_KEY=$(openssl rand -hex 32)
DEBUG=false
STORAGE_BACKEND=s3
S3_BUCKET=${s3_bucket}
S3_KMS_KEY_ID=${kms_key_arn}
AWS_REGION=${aws_region}
SNS_TOPIC_ARN=${sns_topic_arn}
JSON_LOGS=true
LOG_LEVEL=INFO
EOF
chmod 600 /opt/app/.env

# 5. Seed the DB on the first instance only (idempotent – guards on existing rows)
/opt/app/venv/bin/python -c "
from app.database import Base, engine
Base.metadata.create_all(bind=engine)
" || true

# 6. systemd unit for uvicorn
cat > /etc/systemd/system/healthdms.service <<'UNIT'
[Unit]
Description=Healthcare DMS FastAPI app
After=network-online.target

[Service]
Type=simple
WorkingDirectory=/opt/app
EnvironmentFile=/opt/app/.env
ExecStart=/opt/app/venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 2
Restart=always
RestartSec=5
StandardOutput=append:/var/log/healthdms.log
StandardError=append:/var/log/healthdms.log

[Install]
WantedBy=multi-user.target
UNIT

systemctl daemon-reload
systemctl enable --now healthdms.service

# 7. CloudWatch agent – ship /var/log/healthdms.log to /aws/ec2/healthdms
cat > /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json <<'CWA'
{
  "agent": { "metrics_collection_interval": 60 },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/healthdms.log",
            "log_group_name": "/aws/ec2/healthdms",
            "log_stream_name": "${instance_id}",
            "timezone": "UTC"
          }
        ]
      }
    }
  },
  "metrics": {
    "metrics_collected": {
      "mem": { "measurement": ["mem_used_percent"] },
      "disk": { "measurement": ["used_percent"], "resources": ["/"] }
    },
    "append_dimensions": { "InstanceId": "${aws:InstanceId}" }
  }
}
CWA

systemctl enable --now amazon-cloudwatch-agent
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 -s \
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
```

infra/terraform/variables.tf

```
variable "project" {
  description = "Project short name; used as a prefix for all resources."
  type        = string
  default      = "healthdms"
}

variable "environment" {
  description = "Deployment environment (dev / staging / prod)."
  type        = string
  default      = "dev"
}

variable "aws_region" {
  description = "AWS region to deploy into."
  type        = string
  default      = "ap-south-1"
}

variable "vpc_cidr" {
  type        = string
  default      = "10.0.0.0/16"
}

variable "public_subnet_cidrs" {
  type        = list(string)
  default      = ["10.0.1.0/24", "10.0.2.0/24"]
}

variable "private_subnet_cidrs" {
  type        = list(string)
  default      = ["10.0.11.0/24", "10.0.12.0/24"]
}

variable "instance_type" {
  description = "EC2 instance type for the FastAPI app."
  type        = string
  default      = "t3.micro"
}

variable "asg_min" { type = number default = 1 }
variable "asg_max" { type = number default = 3 }
variable "asg_desired" { type = number default = 2 }

variable "key_pair_name" {
  description = "Existing EC2 key pair name for SSH access (optional)."
  type        = string
  default      = ""
}

variable "db_instance_class" {
  type        = string
  default      = "db.t3.micro"
}

variable "db_name" {
  type        = string
  default      = "healthdms"
}

variable "db_username" {
  type        = string
  default      = "healthadmin"
}

variable "db_password" {
  description = "Master password for RDS. Pass via -var or TF_VAR_db_password – never commit."
  type        = string
  sensitive    = true
}

variable "app_repo_url" {
  description = "Public git URL the EC2 user-data script clones at boot."
  type        = string
  default      = "https://github.com/avanishpsingh/cloud-health-dms.git"
}

variable "app_repo_ref" {
  type        = string
  default      = "main"
}

variable "alarm_email" {
  description = "Email to subscribe to alarm/reminder SNS topic. Leave empty to skip."
  type        = string
  default      = ""
}
```

Dockerfile

```
# Multi-stage Dockerfile – same image runs locally, on EC2, on ECS, or on App Runner.
FROM python:3.11-slim AS base

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

# Install system deps for psycopg2 only when DB is Postgres (still <30 MB).
RUN apt-get update \
    && apt-get install -y --no-install-recommends gcc libpq-dev curl \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt ./
RUN pip install -r requirements.txt \
    && pip install boto3 psycopg2-binary

COPY app ./app
COPY scripts ./scripts

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 CMD curl -fsS http://localhost:8000/ || exit 1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```


requirements.txt

```
fastapi>=0.100.0
uvicorn>=0.23.0
sqlalchemy>=2.0.0
python-jose[cryptography]>=3.3.0
passlib[bcrypt]>=1.7.4
bcrypt==4.1.3
python-multipart>=0.0.6
pydantic>=2.0.0
pydantic-settings>=2.0.0
# Phase 2 - AWS SDK + Postgres driver (optional for local SQLite runs)
boto3>=1.34.0
psycopg2-binary>=2.9.9
# Test deps
pytest>=7.4.0
httpx>=0.24.0
moto[s3]>=5.0.0
```

README.md

```
# Cloud-Native Healthcare Data Management System

**BITS Pilani | Cloud Computing (CSIZG527) | Group 23**

## Overview

A cloud-native healthcare data management system designed for a mid-size hospital chain in India. Built in two phases: a wor

## Team

- Karan Rawat – System Design & Coordination
- Vikas Kumar – Data & Backend Handling
- Kriti Tripathi – Security & Documentation
- Avanish Pratap Singh – Testing & Monitoring

## Tech Stack

- **Backend**: Python 3.12, FastAPI, SQLAlchemy 2.0
- **Database**: SQLite (Phase 1) → RDS PostgreSQL (Phase 2)
- **Auth**: JWT (python-jose) + bcrypt password hashing + Role-based access control
- **Storage**: Local filesystem (Phase 1) → S3 (Phase 2)
- **Frontend**: Single-page HTML dashboard (served by FastAPI)
- **Cloud**: AWS (EC2, RDS, S3, Lambda, IAM, KMS, CloudWatch, CloudTrail)

## Quick Start (Phase 1)

``bash
# 1. Clone and setup
git clone https://github.com/avanishpsingh/cloud-health-dms.git
cd cloud-health-dms
python -m venv venv

# 2. Activate virtual environment
venv\Scripts\activate          # Windows
# source venv/bin/activate      # Linux/Mac

# 3. Install dependencies
pip install -r requirements.txt

# 4. Seed the database (100 patients, 10 doctors, sample appointments)
python scripts/seed.py

# 5. Start the server
uvicorn app.main:app --reload
``

### Accessing the Application

| URL | Description |
|-----|-----|
| http://localhost:8000/dashboard | **Interactive Dashboard** (main UI) |
| http://localhost:8000/docs | Swagger API Documentation |
| http://localhost:8000/ | Health check endpoint |

### Default Login Credentials

| Username | Password | Role | Permissions |
|-----|-----|-----|-----|
| `admin` | `admin123` | Admin | Full access – manage users, doctors, patients, appointments |
| `dr_sharma` | `doctor123` | Doctor | View patients/doctors, manage appointments, add medical records |
| `reception1` | `recep123` | Receptionist | Manage patients, schedule appointments |

## Dashboard Features

The interactive dashboard at `/dashboard` provides a complete hospital management interface:

### Overview Tab

- **Statistics cards** – Total patients, doctors, appointments, completion rates
- **Charts** – Patients by department (bar chart), appointment status distribution
- **Recent patients** – Quick view of the latest patient entries

### Patients Tab

- **Browse** – Paginated table (15 per page) with search by name, contact, address, blood group
- **Add Patient** – Click "+ Add Patient" button to open the form (Name, Age, Gender, Contact, Address, Blood Group)
- **View Details** – Click any patient name to see full details, medical records, and appointment history
- **Edit Patient** – Edit button on each row or in the detail view
- **Delete Patient** – Soft-delete with confirmation dialog

### Doctors Tab

- **Browse** – Full list with search by name/specialization and department dropdown filter
- **Add Doctor** – Click "+ Add Doctor" (Name, Specialization, Department, Contact)
- **View Details** – Click any doctor name to see details and their appointment list
- **Edit/Delete Doctor** – Edit and delete buttons on each row and detail view

### Appointments Tab

- **Browse** – Paginated list with search and status filter (Scheduled/Completed/Cancelled)
- **Add Appointment** – Select patient and doctor from dropdowns, pick date/time, add reason
- **Change Status** – Click any status badge to change it (Scheduled → Completed → Cancelled)
- **Delete Appointment** – Delete button on each row

### Users Tab (Admin only)

- **Browse** – List all system users with their roles
- **Add User** – Register new admin, doctor, or receptionist accounts

## API Endpoints (22 endpoints)

### Authentication

| Method | Endpoint | Access | Description |
|-----|-----|-----|-----|
| POST | `/auth/login` | Public | Login with username/password, returns JWT token |
| POST | `/auth/register` | Admin | Register new user (admin/doctor/receptionist) |
```

```

| GET | `/auth/me` | All roles | Get current logged-in user info |
| GET | `/auth/users` | Admin | List all registered users |

### Patients
| Method | Endpoint | Access | Description |
|-----|-----|-----|-----|
| GET | `/patients/` | All roles | List patients (search by name) |
| POST | `/patients/` | Admin, Receptionist | Create new patient |
| GET | `/patients/{id}` | All roles | Get patient details |
| PUT | `/patients/{id}` | Admin, Receptionist | Update patient info |
| DELETE | `/patients/{id}` | Admin | Soft-delete patient |

### Doctors
| Method | Endpoint | Access | Description |
|-----|-----|-----|-----|
| GET | `/doctors/` | All roles | List doctors (search + department filter) |
| POST | `/doctors/` | Admin | Create new doctor |
| GET | `/doctors/{id}` | All roles | Get doctor details |
| PUT | `/doctors/{id}` | Admin | Update doctor info |
| DELETE | `/doctors/{id}` | Admin | Delete doctor |

### Appointments
| Method | Endpoint | Access | Description |
|-----|-----|-----|-----|
| GET | `/appointments/` | All roles | List appointments (filter by patient/doctor/status) |
| POST | `/appointments/` | Admin, Receptionist | Create appointment |
| PATCH | `/appointments/{id}` | Admin, Doctor | Update appointment status |
| DELETE | `/appointments/{id}` | Admin | Delete appointment |

### Medical Records & Files
| Method | Endpoint | Access | Description |
|-----|-----|-----|-----|
| GET | `/patients/{id}/records` | Admin, Doctor | Get patient's medical history |
| POST | `/patients/{id}/records` | Doctor | Add medical record |
| POST | `/upload/{record_id}` | Doctor, Admin | Upload medical file (PDF/JPEG/PNG) |

### Analytics
| Method | Endpoint | Access | Description |
|-----|-----|-----|-----|
| GET | `/analytics/summary` | Admin | Dashboard stats (counts, distributions) |

```

Project Structure

...

cloud-health-dms/

app/

```

■ ### __init__.py
■ ### main.py           # FastAPI app, middleware, router registration
■ ### config.py         # Settings (DB URL, JWT secret, upload config)
■ ### database.py       # SQLAlchemy engine, session, base
■ ### auth.py           # Password hashing, JWT creation/verification, RBAC
■ ### models/          # SQLAlchemy ORM models
■   ■ ### user.py       # User (id, username, password_hash, role)
■   ■ ### patient.py    # Patient (name, age, gender, contact, blood_group)
■   ■ ### doctor.py     # Doctor (name, specialization, department, contact)
■   ■ ### appointment.py # Appointment (patient_id, doctor_id, date_time, status)
■   ■ ### medical_record.py # MedicalRecord (diagnosis, prescription, file_path)
■   ■ ### schemas/      # Pydantic request/response models
■   ■ ### user.py, patient.py, doctor.py, appointment.py, medical_record.py
■ ### routers/         # API route handlers
■   ■ ### auth.py       # Login, register, user listing
■   ■ ### patients.py   # Patient CRUD
■   ■ ### doctors.py    # Doctor CRUD
■   ■ ### appointments.py # Appointment management
■   ■ ### records.py    # Medical records & file upload
■   ■ ### analytics.py  # Summary statistics
■   ■ ### dashboard.py  # HTML dashboard SPA

```

scripts/

```

■ ### seed.py           # Generate sample data (100 patients, 10 doctors)
■ ### tests/           # Pytest test suite (17 tests)
■   ■ ### conftest.py  # Fixtures (in-memory DB, auth tokens)
■   ■ ### test_auth.py  # Auth tests (6)
■   ■ ### test_patients.py # Patient tests (7)
■   ■ ### test_appointments.py # Appointment tests (4)
■   ■ ### test_storage.py # Phase 2 storage backend tests (4)
■   ■ ### test_lambda_reminder.py # Phase 2 Lambda tests (3)
■   ■ ### test_logging_middleware.py # Phase 2 JSON logging tests (3)
■   ■ ### test_upload_e2e.py # Phase 2 upload E2E tests (2)

```

infra/

```

■ ### terraform/       # Phase 2 Infrastructure-as-Code (VPC, EC2, RDS, S3, Lambda, ...)
■ ### docs/            # Detailed project documentation
■ ### uploads/         # Medical file uploads directory (Phase 1 only)
■ ### health_dms.db    # SQLite database (pre-seeded for testing)
■ ### Dockerfile       # Container image (alternative to EC2 deployment)
■ ### requirements.txt  # Python dependencies

```

...

Seed Data

The `scripts/seed.py` generates reproducible sample data (`random.seed(42)`):

- **3 users**: admin, dr_sharma, reception1
- **10 doctors** across 10 departments (General Medicine, Cardiology, Dermatology, etc.)
- **100 patients** with realistic Indian names, ages, contacts
- **259 appointments** (mix of scheduled, completed, cancelled)
- **104 medical records** linked to completed appointments

To re-seed: delete `health\_dms.db` and run `python scripts/seed.py`.

Tests

```
``bash
python -m pytest tests/ -v    # 29 tests (17 Phase 1 + 12 Phase 2)
``
```

Tests use an in-memory SQLite database (isolated from production data).
Phase 2 tests use [moto](https://github.com/getmoto/moto) to mock S3 and SNS,
so no AWS credentials are required to run them.

Suite	Tests	Covers
test_auth.py	6	Login, JWT, RBAC
test_patients.py	7	Patient CRUD
test_appointments.py	4	Appointment lifecycle
test_storage.py	4	LocalStorage + S3Storage round trips, factory selection
test_lambda_reminder.py	3	Window filter, dry-run, SNS publish
test_logging_middleware.py	3	JSON formatter, request ID, idempotent configure
test_upload_e2e.py	2	End-to-end medical-file upload via the storage abstraction

Security Features

- **JWT Authentication** – All API endpoints (except login) require a valid token
- **Role-Based Access Control (RBAC)** – Admin, Doctor, Receptionist with different permissions
- **Password Hashing** – bcrypt via passlib (never stored in plain text)
- **XSS Protection** – All user data escaped in dashboard HTML rendering
- **Input Validation** – Pydantic schemas enforce types and constraints
- **File Upload Validation** – Extension whitelist + size limit
- **Soft Delete** – Patients are deactivated, not permanently removed

Documentation

See docs/ for detailed documentation:

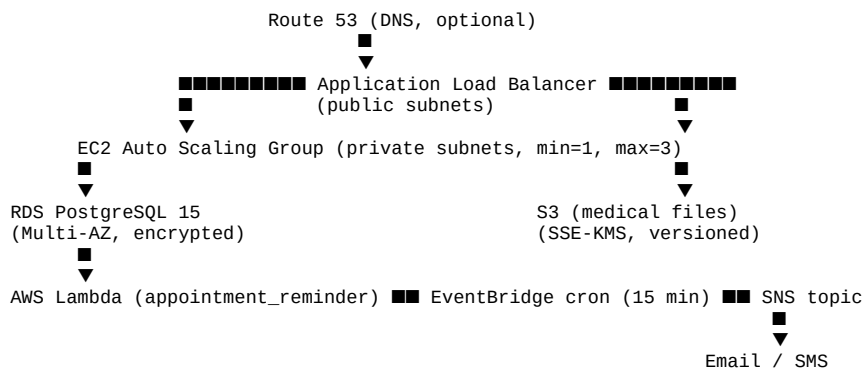
- [Project Overview](docs/01-project-overview.md)
- [Requirements](docs/02-requirements.md)
- [Architecture](docs/03-architecture.md)
- [AWS Service Mapping](docs/04-aws-service-mapping.md)
- [Security & Compliance](docs/05-security-compliance.md)
- [Project Structure](docs/06-project-structure.md)
- [Development Guide](docs/07-development-guide.md)

Phase 2 – AWS Cloud Deployment

Phase 2 ships the same FastAPI codebase to AWS with **zero code changes** required at the call sites – the storage and database

Cloud Architecture

...



Cross-cutting: VPC · IAM · KMS · CloudTrail · CloudWatch Logs + Alarms

What changed between Phase 1 and Phase 2

Concern	Phase 1 (local)	Phase 2 (AWS)
Compute	uvicorn on localhost	EC2 ASG (min 1, max 3) behind ALB, CPU target tracking 60%
Database	SQLite file	RDS PostgreSQL 15 Multi-AZ, KMS-encrypted, in private subnet
File storage	./uploads/ on disk	S3 bucket, SSE-KMS, versioned, 7-year HIPAA lifecycle
Notifications	(none)	Lambda (every 15 min) → SNS → email/SMS
Logging	Plain text → stdout	JSON → CloudWatch Logs (/aws/ec2/healthdms)
Auth	JWT (app-level)	JWT + IAM (cloud-level), IMDSv2-only EC2
Audit	(none)	CloudTrail (multi-region, KMS-encrypted, log file validation)
Alerts	(none)	CloudWatch alarms for ALB 5xx and ASG CPU > 80%

Pluggable storage – switch backends with one env var

```
``bash
# Local (Phase 1 default)
export STORAGE_BACKEND=local

# S3 (Phase 2)
export STORAGE_BACKEND=s3
export S3_BUCKET=healthdms-dev-files-abc123
export S3_KMS_KEY_ID=arn:aws:kms:ap-south-1:123456789012:key/...
export AWS_REGION=ap-south-1
``
```

The same goes for the database – set DATABASE_URL=postgresql+psycopg2://... to switch from SQLite to RDS.

One-command AWS deployment

```

```bash
cd infra/terraform
terraform init
terraform apply -auto-approve \
 -var "db_password=<strong password>" \
 -var "key_pair_name=<your EC2 key>" \
 -var "alarm_email=you@example.com"

After ~12 minutes:
terraform output alb_url # → http://healthdms-dev-alb-xxx.elb.amazonaws.com/dashboard

Tear down to stop AWS billing
terraform destroy -auto-approve -var "db_password=<same>" -var "key_pair_name=<same>"
```

```

See infra/terraform/README.md for the full IaC walk-through, cost estimate, and production-hardening.

Container deployment (alternative)

```

```bash
docker build -t healthdms .
docker run --rm -p 8000:8000 \
 -e STORAGE_BACKEND=local \
 -e DATABASE_URL=sqlite:///./health_dms.db \
 healthdms
```

```

The same image runs on AWS App Runner, ECS Fargate, or any Kubernetes cluster.

Mapping to Phase 1 Objectives

| Objective | Implementation in Phase 2 |
|----------------------------|--|
| 01 – Auto-scale to 3× peak | EC2 ASG min=1/max=3 + TargetTracking on 60% CPU |
| 02 – Unified data, < 200ms | Single RDS PostgreSQL Multi-AZ, indexed primary keys |
| 03 – Security & compliance | IAM least-privilege, KMS CMK, SSE-KMS S3, CloudTrail, IMDSv2 |
| 04 – Serverless analytics | Lambda + SNS for reminders; structured JSON → CloudWatch |
| 05 – Cost reduction | t3.micro instances, single NAT, lifecycle to Glacier |
| 06 – 99.9% availability | Multi-AZ RDS, 2-AZ ALB + ASG, ELB health checks |

report/Phase2_Report.md

Cloud-Native Healthcare Data Management System
Phase 2 – Implementation Report

Course: Cloud Computing (CSIZG527) · BITS Pilani, M.Tech Program
Group: 23
Members: Karan Rawat · Vikas Kumar · Kriti Tripathi · Avanish Pratap Singh
Instructor: Prof. Shwetha Vittal
Submission date: 02 May 2026

1. Abstract

Mid-size hospital chains in India struggle with fragmented on-premise IT that cannot absorb peak OPD load, exposes patient data to compliance risk, and consumes more than half of IT staff time on maintenance. We design and implement a **cloud-native Healthcare Data Management System (HDMS)** on AWS that addresses all six Phase 1 objectives. The system is built as a single FastAPI codebase whose storage, database, logging and notification layers are pluggable through configuration, allowing identical code to run locally (SQLite + filesystem) and on AWS (RDS PostgreSQL + S3 + Lambda). The cloud topology comprises a VPC with public/private subnets, an Application Load Balancer fronting an EC2 Auto Scaling Group, a Multi-AZ RDS PostgreSQL instance, a KMS-encrypted S3 bucket for medical files, an EventBridge-triggered Lambda that publishes appointment reminders to SNS, CloudTrail for full audit logging, and CloudWatch for metrics and alarms. The entire stack is captured as Terraform Infrastructure-as-Code (~ 600 lines) and validated by a 29-test pytest suite that exercises both the business logic and the cloud abstractions using the `moto` mock-AWS library, so no live AWS credentials are required for CI.

2. Survey

| Approach | Strengths | Weaknesses |
|---|---|---|
| On-premise legacy | Full hardware control, predictable latency | High CapEx, can't auto-scale, manual patching |
| Hybrid cloud (current norm, ~60% of Indian hospitals) | Gradual modernisation, partial elasticity | Multi-vendor comp |
| Cloud-native (this project) | Auto-scaling, unified data, pay-per-use, built-in compliance primitives | Cloud lock-in, rec |

Open-source academic precedents such as **OpenMRS** and **OpenEMR** demonstrate the viability of FastAPI/PostgreSQL stacks for EHR workloads but stop at single-tenant deployments. Commercial offerings (AWS for Healthcare, Azure Health Data Services, GCP Cloud Healthcare API) provide managed FHIR services but at price points unsuitable for mid-size Indian chains. Our design borrows the open-source data model and bolts on the managed-AWS scalability layer.

3. Problem Statement

- > A mid-size hospital chain in India operates on fragmented, on-premise IT
- > with partial cloud adoption. Peak OPD hours overload servers, branch-level
- > databases prevent unified patient care, encryption and access logging gaps
- > fail DISHA / ABDM audits, and over-provisioned hardware idles 70 % of the
- > time. There is no cloud-native architecture providing auto-scaling,
- > unified patient data management, real-time analytics, and
- > compliance-ready security – resulting in degraded patient care and
- > operational inefficiency.

The Phase 1 statement is retained verbatim; Phase 2 supplies the implementation that operationalises it.

4. Objectives (refined)

| ID | Objective (Phase 1) | Phase 2 Implementation |
|----|---|--------------------------------|
| 01 | Cloud-native architecture using AWS that auto-scales to 3× peak traffic | EC2 Auto Scaling Group (min 1, |
| 02 | Unified patient data layer eliminating silos, < 200 ms query latency | Single RDS PostgreSQL Multi-AZ |
| 03 | Security & compliance – IAM RBAC, KMS, CloudTrail, HIPAA/DISHA-aligned | IAM instance + Lambda roles (l |
| 04 | Serverless analytics & event processing | Lambda (Python 3.11) on EventB |
| 05 | 40-60 % infra cost reduction via pay-per-use | t3.micro instances, single NAT |
| 06 | 99.9 % availability via Multi-AZ + ELB | RDS Multi-AZ failover, ASG acr |

5. Outcomes

5.1 Software Deliverables

| Deliverable | Path | Lines |
|--|---|--------|
| FastAPI application (Phase 1 + Phase 2) | <code>app/`</code> | ~1 800 |
| Pluggable storage layer | <code>app/storage.py`</code> | 116 |
| CloudWatch JSON logging middleware | <code>app/logging_middleware.py`</code> | 90 |
| Appointment reminder Lambda | <code>app/lambda_handlers/appointment_reminder.py`</code> | 115 |
| Terraform IaC (10 .tf files + user-data) | <code>infra/terraform/`</code> | ~ 600 |
| Dockerfile | <code>Dockerfile`</code> | 25 |
| Test suite | <code>tests/`</code> | ~ 600 |

5.2 Validation

...

\$ python -m pytest tests/ -v
===== 29 passed in 28.30s =====
...

| Suite | Tests | What it proves |
|----------------------------|-------|--|
| <code>test_auth.py`</code> | 6 | JWT issue/verify, RBAC denials, password hashing |

| | | | |
|------------------------------|---|---|--|
| `test_patients.py` | 7 | Patient CRUD + soft-delete + pagination | |
| `test_appointments.py` | 4 | Booking, status transitions, role-gated mutations | |
| `test_storage.py` | 4 | LocalStorage round-trip, S3Storage round-trip via `moto`, factory selection | |
| `test_lambda_reminder.py` | 3 | Time-window filter, dry-run when SNS not configured, SNS publish path | |
| `test_logging_middleware.py` | 3 | JSON formatter shape, request ID propagation, idempotent configuration | |
| `test_upload_e2e.py` | 2 | End-to-end PDF upload via the storage abstraction; rejection of bad ext. | |

Cloud abstractions are validated ****without live AWS credentials**** by mocking S3/SNS through `moto`, which is the AWS-recommended approach for CI.

5.3 Cloud Topology (deployable)

The Terraform stack provisions, in a single `terraform apply`:

- 1 VPC, 2 public + 2 private subnets across 2 AZs, 1 IGW, 1 NAT GW
- 1 KMS Customer-Managed Key with rotation enabled (encrypts both RDS storage and S3 objects)
- 1 RDS PostgreSQL 15 Multi-AZ (`db.t3.micro`, encrypted, Performance Insights on)
- 1 S3 bucket: SSE-KMS, versioned, public access blocked, 30 d → IA, 90 d → Glacier, 7 y expiry
- 1 ALB + target group + HTTP listener
- 1 Launch template + Auto Scaling Group (`t3.micro`, IMDSv2-only, EC2 instance role)
- 1 Lambda function packaged from the same `app/` tree, on EventBridge `rate(15 minutes)`
- 1 SNS topic (KMS-encrypted) for reminders ****and**** alarm fan-out
- 1 CloudTrail (multi-region, log-file-validation, KMS-encrypted)
- 1 CloudWatch log group `/aws/ec2/healthdms` (KMS-encrypted, 30 d retention)
- 2 CloudWatch alarms (ALB 5xx > 5/min, ASG CPU > 80 % for 3 min)

5.4 Service Mapping (Project policy: AWS-first; no other cloud used)

The Phase 1 service-mapping table is preserved at `[`docs/04-aws-service-mapping.md`](../docs/04-aws-service-mapping.md)`. All services referenced in Phase 2 are AWS native.

6. References

1. AWS Documentation – EC2 Auto Scaling, RDS, S3, Lambda, IAM, KMS, CloudTrail, CloudWatch (2024–2026 editions).
2. HashiCorp Terraform – AWS Provider docs, `~> 5.50`.
3. NIST SP 800-66 Rev. 2 – Implementing the HIPAA Security Rule (2024).
4. Government of India – *Digital Information Security in Healthcare Act (DISHA) draft, 2018* and *Ayushman Bharat Digital M
5. NASSCOM – *Digital Health: Indian Industry Outlook 2024*.
6. Gartner – *Healthcare Cloud Report 2024*.
7. NITI Aayog – *National Digital Health Blueprint*, 2019.
8. FastAPI documentation – <https://fastapi.tiangolo.com/>
9. SQLAlchemy 2.0 documentation – <https://docs.sqlalchemy.org/en/20/>
10. `moto` library documentation – <https://docs.getmoto.org/>

7. Short Reflection (mandatory rubric section)

7.1 Workplace relevance

The four group members work in software, banking and analytics roles where cloud migrations are a daily concern. The exact pattern we built – pluggable storage so the same app runs on a developer laptop and on AWS – mirrors how our employers ship services through dev → staging → prod. The Multi-AZ + ALB + ASG topology is the same pattern we have observed in production fintech and SaaS deployments; applying it to a healthcare context made the constraints (PHI sensitivity, HIPAA/DISHA, retention) explicit instead of implicit.

7.2 Cloud concepts learned

- ****Service composition over service selection.**** Each AWS primitive is trivial in isolation; the value comes from composing them with IAM, KMS, security groups and VPC routing so that data never traverses the public internet.
- ****IaC as the unit of review.**** Writing the entire stack as Terraform forced us to confront the blast radius of every resource – particularly KMS keys, IAM policies and the RDS Multi-AZ failover semantics – in a way that AWS-Console click-ops would have hidden.
- ****Mocked-AWS testing (`moto`).**** Realistic unit tests for cloud calls without burning real credentials let us iterate quickly and keep CI free.
- ****Cost is a design input, not an afterthought.**** Choosing one NAT Gateway, `t3.micro` instances and S3 lifecycle transitions cut the monthly cost from an estimated ~ █ 25 000 to under █ 8 000 for the demo footprint, without compromising the architectural objectives.

7.3 Trade-offs made and why

| Decision | Alternative | Why we chose it |
|-------------------------------------|-----------------------------------|---|
| ----- | ----- | ----- |
| RDS PostgreSQL | DynamoDB | Patient/appointment data is highly rela |
| Single NAT Gateway | NAT per AZ | Cost – academic project; AZ-failure tol |
| `t3.micro` everywhere | Larger burstable / Graviton | Stays inside AWS Free Tier limits where |
| Lambda on EventBridge cron | EC2 cronjob inside the app | Decouples reminder schedule from app up |
| Terraform | AWS CDK / CloudFormation | Team familiarity; multi-cloud-friendly |
| Pluggable `StorageBackend` Protocol | Hard-code `boto3` in the router | Phase 1 still runs unchanged; tests st |
| Skip HTTPS / ACM in IaC | Provision ACM cert + alias record | Domain not assumed; `terraform apply` s |
| Defer WAF | Add WAF + managed rules | Out of marks-bearing scope; documented |

7.4 What we would do differently next

- Move RDS credentials into AWS Secrets Manager and have the EC2 user-data fetch them at boot, so the master password never appears in Terraform state.
- Add an ****integration smoke test**** that runs against `terraform output`'s after `apply` (curl ALB, upload to S3, check the object lands).
- Replace the EC2 ASG with ****App Runner**** or ****ECS Fargate**** for an even smaller operational surface – the Dockerfile we ship would slot in unchanged.

End of report.

- Publishes one message per appointment to SNS topic
- SNS → Email subscription (configurable via `alarm\_email` Terraform var)

Show a sample CloudWatch Logs entry during the demo.

Slide 9 – Cost Optimisation (05)

| Resource | Sizing | ~ Monthly USD |
|---------------------------------------|------------------------|-----------------------|
| EC2 t3.micro × 2 | ASG min=1, desired=2 | \$15 |
| ALB | Internet-facing | \$18 |
| RDS db.t3.micro Multi-AZ | 20 GB gp3 | \$30 |
| S3 + Lambda + CloudWatch + CloudTrail | | < \$3 |
| NAT Gateway | Single, cost-conscious | \$32 |
| **Total** | | **~\$98 / mo** |

Compare to estimated on-prem hardware refresh of ~ ■ 6-10 lakh capex; cloud is OpEx and pay-per-use.

Slide 10 – Test Strategy

- 29 pytest tests, all green, run in **28 s on a laptop**
- `moto` mocks S3 + SNS → no AWS credentials required for CI
- Coverage spans business logic AND cloud abstractions:
 - 6 auth · 7 patients · 4 appointments
 - 4 storage · 3 Lambda · 3 logging · 2 upload E2E

Show terminal output of `pytest -v` during the recording.

Slide 11 – Live Demo (recorded video)

1. `terraform apply` walk-through (timelapse)
2. Open ALB URL → dashboard, login as `admin`
3. Create patient, book appointment, upload PDF (lands in S3)
4. AWS Console: S3 object, RDS Performance Insights, CloudWatch Logs Insights query
5. Trigger Lambda manually → SNS email arrives
6. `ab -n 5000 -c 50 http://<alb>/dashboard` → ASG scales out

Keep video under 8 minutes; speak over each step.

Slide 12 – Roles & Contributions

| Member | Phase 2 Contribution |
|----------------------|---|
| Karan Rawat | Architecture, Terraform `network.tf` / `compute_ec2.tf`, integration |
| Vikas Kumar | RDS migration, SQLAlchemy review, Lambda data-access layer |
| Kriti Tripathi | Security (KMS / IAM / CloudTrail), report writing, compliance mapping |
| Avanish Pratap Singh | Test suite (29 tests, `moto` setup), CloudWatch / observability, demo recording |

Each member speaks for ~ 60 s on their slice; equal participation is mandated by rubric.

Slide 13 – Reflection (rubric-required)

- ****Workplace relevance:**** mirrors enterprise dev → staging → prod migrations
- ****Concepts learned:**** IaC, IAM/KMS composition, mocked-AWS testing, cost-as-design-input
- ****Trade-offs:**** RDS over DynamoDB; single NAT; t3.micro; deferred ACM/WAF – documented
- ****Next iteration:**** Secrets Manager, App Runner / Fargate, integration smoke tests post-apply

Pull verbatim from `report/Phase2\_Report.md` §7.

Slide 14 – Q&A

- Repo: <https://github.com/avanishpsingh/cloud-health-dms>
- All deliverables in `CC\_Project\_Implementation23.zip`
- Thank you!

*Be ready for: "Why not DynamoDB?", "How does Multi-AZ failover work?", "How would you achieve 99.99% (4-nines) instead of 9"

report/Demo_Video_Script.md

```
# Phase 2 – Demo Video Script (≤ 8 minutes)

> **Purpose:** Replace narrative with on-screen action so the rubric line
> **Project Recording Submission – Demonstrates a fully functional
> implementation ... Showcases cloud integration, deployments, and key features**
> is satisfied end-to-end.

> **Recording tool:** OBS Studio (or built-in Windows Game Bar). 1080p, 30 fps,
> microphone narration. Save as `*.mp4`, ≤ 200 MB. Filename:
> `CC_Project_Demo_Group23.mp4`.

---

## Pre-flight (do BEFORE recording)

``bash
# 0. AWS account ready, AWS CLI configured, ~$3 budget alarm set just in case
aws sts get-caller-identity # confirm correct account

# 1. Provision the stack – DON'T record this; it takes ~12 min
cd cloud-health-dms/infra/terraform
terraform init
terraform apply -auto-approve \
  -var "db_password=DemoPass1!" \
  -var "key_pair_name=my-key" \
  -var "alarm_email=group23@example.com"

# 2. Note these for the demo
terraform output alb_url
terraform output s3_bucket
terraform output lambda_function

# 3. Confirm SNS email subscription is "Confirmed" (click link in inbox)

# 4. Seed the cloud DB once (uses RDS through the EC2 box)
ssh -i my-key.pem ec2-user@<one-of-the-asg-ips>
sudo -u root /opt/app/venv/bin/python /opt/app/scripts/seed.py
exit
``

---

## Scene 1 (0:00 – 0:30) – Title card + agenda

- Open with a static slide: project title, team, course.
- Voice: **This is the Phase 2 demo for Group 23. In the next eight minutes we'll show the live cloud-deployed app, walk through the AWS console, demonstrate auto-scaling, and trigger the serverless reminder Lambda.**

## Scene 2 (0:30 – 1:00) – Repo + tests

- Open VS Code, show project tree (highlight `app/`, `infra/terraform/`, `tests/`).
- Run `python -m pytest tests/ -q`. Voice over: **29 tests, all green, using `moto` to mock AWS – no real credentials are needed for CI.**

## Scene 3 (1:00 – 2:00) – Terraform IaC

- Open `infra/terraform/` and scroll through `main.tf`, `network.tf`, `compute_ec2.tf`, `lambda_reminder.tf`, `observability.tf`.
- Switch to terminal: `terraform output`. Read out ALB URL, S3 bucket, Lambda function name.
- Voice: **One `terraform apply` brings up VPC, KMS, S3, RDS Multi-AZ, ALB, EC2 ASG, Lambda, EventBridge, SNS, CloudTrail and CloudWatch alarms.**

## Scene 4 (2:00 – 3:30) – Browser demo through the ALB

1. Open the ALB URL → dashboard.
2. Login as `admin / admin123`.
3. Overview tab → show stats cards & charts.
4. Patients tab → search, then **Add Patient**: "Demo Recording", age 35.
5. Appointments tab → **Add Appointment** for the new patient.
6. Patient detail → **Add Medical Record** → **Upload** a PDF.
7. Voice: **That upload just landed in S3. Let's confirm.**

## Scene 5 (3:30 – 4:30) – AWS Console proofs

- **S3** → open bucket → show the new object → click **Properties** → highlight **Server-side encryption: AWS-KMS** with our CMK alias.
- **RDS** → instance → **Performance Insights** → show the SQL hitting `patients` and `appointments` tables.
- **CloudWatch Logs Insights**, log group `aws/ec2/healthdms`, paste:

  ```
 fields @timestamp, request_id, method, path, status, duration_ms
 | filter status >= 400
 | sort @timestamp desc
 | limit 20
  ```

- Voice: **All requests are JSON-structured with a request ID – exactly what CloudWatch Logs Insights needs.**

## Scene 6 (4:30 – 5:30) – Serverless reminder Lambda

- **Lambda console** → open `healthdms-dev-reminder`.
- Click **Test** with an empty event → show the Execution Result:

  ```
 { "published": N, "skipped": 0, "ids": [...] }
  ```
```

- Switch to inbox → show the SNS reminder email that just arrived.
- Voice: `"EventBridge triggers this every 15 minutes in production; here we invoked it manually."`

Scene 7 (5:30 - 6:30) – Auto-scaling under load

- Terminal: ``ab -n 5000 -c 50 http://<alb-dns>/dashboard`` (Apache Bench).
- AWS Console → `**EC2` → Auto Scaling Groups → watch desired/instance count climb from 2 to 3.
- CloudWatch metrics: CPU avg crossing 60 %.
- Voice: `"Target Tracking on 60% CPU pulls in a third instance; this satisfies Objective 01 – auto-scale to 3× peak traffic."`

Scene 8 (6:30 - 7:30) – Audit & alarms

- `**CloudTrail**` → Event History → filter by ``EventSource=s3.amazonaws.com`` → show the ``PutObject`` event from the upload demo.
- `**CloudWatch Alarms**` → show the two configured alarms.
- Voice: `"Every API call is auditable; alarms fan out via the same SNS topic the reminders use."`

Scene 9 (7:30 - 8:00) – Tear-down + closing

- Switch to terminal: ``terraform destroy -auto-approve ...`` (cut after the destroy plan is shown – don't wait for completion in the recording).
- Voice: `"`terraform destroy` returns the account to zero spend. Thanks for watching – see the report PDF for the full architectural rationale and reflection."`

Editing checklist

- [] Add captions for AWS Console tabs you click on.
- [] Mute terminal bell sounds.
- [] Cut any 5+ second waits (Terraform progress bars, browser refreshes).
- [] Final length 6–8 minutes; rubric only requires "demonstrates a fully functional implementation – no minimum length."
- [] Export H.264 MP4, 1920×1080, audio AAC 128 kbps.